BIA Business and Information Systems Architecture
MGMT-6134-(31)-25F Capstone Project

HEARTLINK: Heart rate Tracking and Analyst

Final

Professor Vinnie Moraes & Georgi Ivanov

Name – Student ID	Email
Guan, Ivan – 1289006 (Captain)	y_guan251701@fanshaweonline.ca
Diden, Gbubemi - 1205960	g_diden@fanshaweonline.ca
Dinh, Patrick - 1260577	n_dinh240521@fanshaweonline.ca

Version 1.0 Dec 10, 2025

Index

1. Project Charter 5

1.1 Introduction 5

1.2 Deliverables 5

1.3 Key Stakeholders 5

1.4 Milestones Summary 6

1.5 Gantt Chart and Critical Path 7

2. Problem Statement 9

3. Requirements Gathering 10

3.1 Functional Requirements 10

3.2 Non-Functional Requirements 10

3.3 Assumptions, Constraints, and Acceptance Criteria 11

4. Web Search 13

4.1 Hardware Development Technologies and Frameworks 13

4.2 Maintenance and Version Control 14

4.3 Frontend Technologies 15

4.4 User Interface (UI) and User Experience (UX) Design 15

4.5 Data Engineering Pipeline 16

5. Data Dictionary 18

5.1 Hardware Layer Data Dictionary 18

5.2 Front-End Layer Data Dictionary 20

5.3 Cloud Layer Data Dictionary 23

6. Embedded Services and Third-Party Integrations 26

6.1 ESP-IDF 26

6.2 Google Cloud Pub/Sub 27

6.3 Google Cloud Storage & Google Big Query 27

6.4 Firebase Authentication 28
6.5 Looker Studio 28
6.6 Google Cloud Run 28
7. UI and UX Prototypes 30
7.1 Hardware Prototype 30
7.2 Front-end Prototype 30
8. Project Design 37
8.1 System Architecture 37
8.2 Hardware UML Diagram 38
8.3 Front-End UML Diagrams 40
8.4 Cloud Layer UML Diagrams 42
8.5 Entity Relationship Diagram 43
9. API Functions 44
9.1 Data Ingestion API (Device-to-Cloud) 44
9.2 Device Control API (Device Polling) 44
9.3 User Control API (Frontend-to-Cloud) 45
10. Test Cases 47
10.1 Scope and Methodology 47
10.2 Test Case Classification 47
10.3 Test Cases 49
11. Coding Standards 55
11.1 General Principles 55
11.2 Naming Conventions 55
11.3 Code Formatting 55
11.4 Documentation and Comments 55
11.5 Language-Specific Standards 56

12. Deployment Guide and Source Code 57

12.1 Deployment Diagram 57

12.2 Deployment Guide 58

12.3 Source Code of Hardware 60

13 Recommendations 61

13.1 Hardware Layer 61

13.2 Cloud Layer 61

13.3 Front-end Layer 61

14 Summary 63

15 References 64

1.Project Charter

1.1 Introduction

This project presents the design and implementation of an Internet of Things (IoT) based health monitoring device that can measure and analyze a user's vital signs in real time. The system integrates a MAX30102 pulse oximeter and heart rate sensor with an ESP32 board, which transmits the collected physiological data to a cloud server.

On the cloud side, the data is processed and analyzed using AI-based algorithms to detect patterns related to heart rate variability (HRV) and potential health anomalies. The analyzed results are then visualized through a web-based user interface, providing users with intuitive charts and insights into their cardiovascular health.

1.2 Deliverables

1.2.1 Core Requirements

- ESP32 + MAX30102 hardware setup with proper wiring and configuration
- Firmware code for ESP32 to collect heart rate and HRV data
- Cloud integration for real-time data storage
- Web-based dashboard for real-time heart rate visualization
- Documentation (system design, user guide, source code with comments)

1.2.2 Enhanced Functionality

- AI-based HRV analysis module providing stress/fatigue indicators
- Data logging and history tracking with cloud database integration
- Interactive dashboard with charts and trend visualization
- Basic data security (encrypted communication, user authentication)

1.2.3 Optional Extensions

- Sleep detection insights from HR/HRV data
- SpO_2 (blood oxygen) detection using MAX30102
- Multi-user account support for personalized tracking

1.2.4 Optional Extensions

- Cloud-based ML model for advanced anomaly detection

1.3 Key Stakeholders

This project relies on the collaboration and support of multiple stakeholders. Each contributes unique expertise and resources that are essential to achieving the project's objectives. The primary stakeholders include:

1.3.1 End Users (General Users / Students / Volunteers):

- Individuals who will test and interact with the prototype device.
- Provide feedback on usability, comfort, and clarity of health insights.

1.3.2 Cloud Service Providers:

- Provide infrastructure for real-time data storage, processing, and availability.
- Ensure scalability and reliability of the backend system.

1.3.3 Future Investors / Industry Partners (Potential Stakeholders):

- Could be interested in extending the system for commercialization or further research.
- Evaluate the feasibility of scaling the project into a practical product.

1.3.4 Mentors:

- Vinnie Moraes & Georgi Ivanov:

  Provides guidance, strategic advice, and oversight to ensure the project meets academic and professional standards.

1.3.5 Project Team:

- Ivan Guan (Project Manager/Embedded Systems Developer):
  - Leads the project, manages timelines, and coordinates tasks.
  - Designs and programs the ESP32 with the MAX30102 sensor to ensure accurate data collection and reliable device performance.
- Nguyen Hong Thien Phuc Dinh (Data Engineer / AI Developer):
  - Handles cloud integration and data pipelines.
  - Develops AI/HRV analysis modules to interpret health data and generate meaningful insights for end-users.
- Gbubemi Diden (UI/UX Designer/Frontend Developer):
  - Designs and builds the web dashboard.
  - Focuses on creating a responsive, user-friendly interface.

1.4 Milestones Summary

Event	Description	Start date	Due date	Status
Milestone 1	Planning & Requirements	2025-09-03	2025-09-29	Completed
Milestone 2 & 3	System Design	2025-09-30	2025-10-10	Completed
Milestone 4	Development	2025-10-09	2025-10-31	Completed
Milestone 4	Testing & Validation	2025-10-31	2025-11-19	Completed
Final	Finalization & Reporting	2025-11-19	2025-12-05	Completed

Table 1: Milestone summary and status

1.5 Gantt Chart and Critical Path

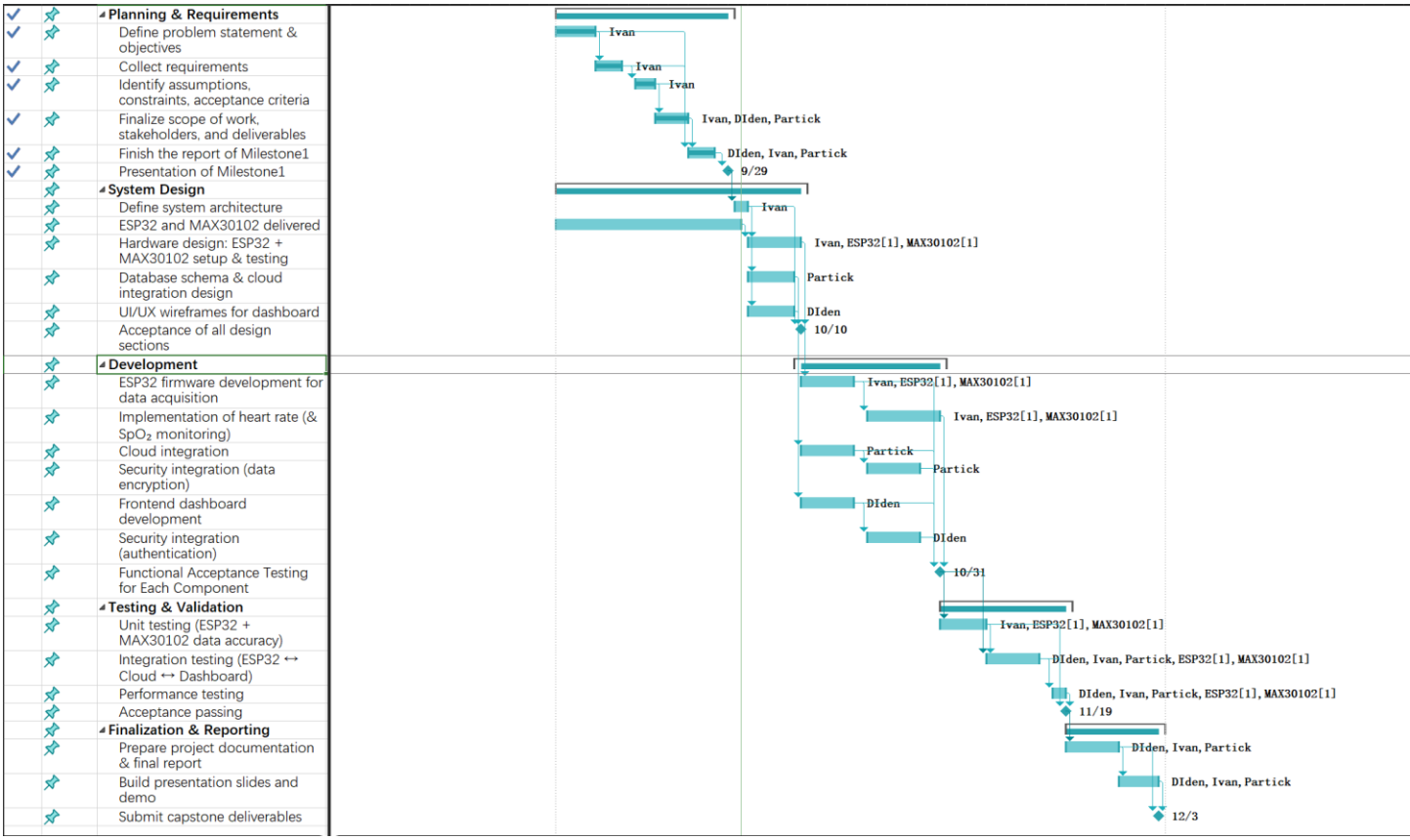

Figure 1: Gantt Chart View

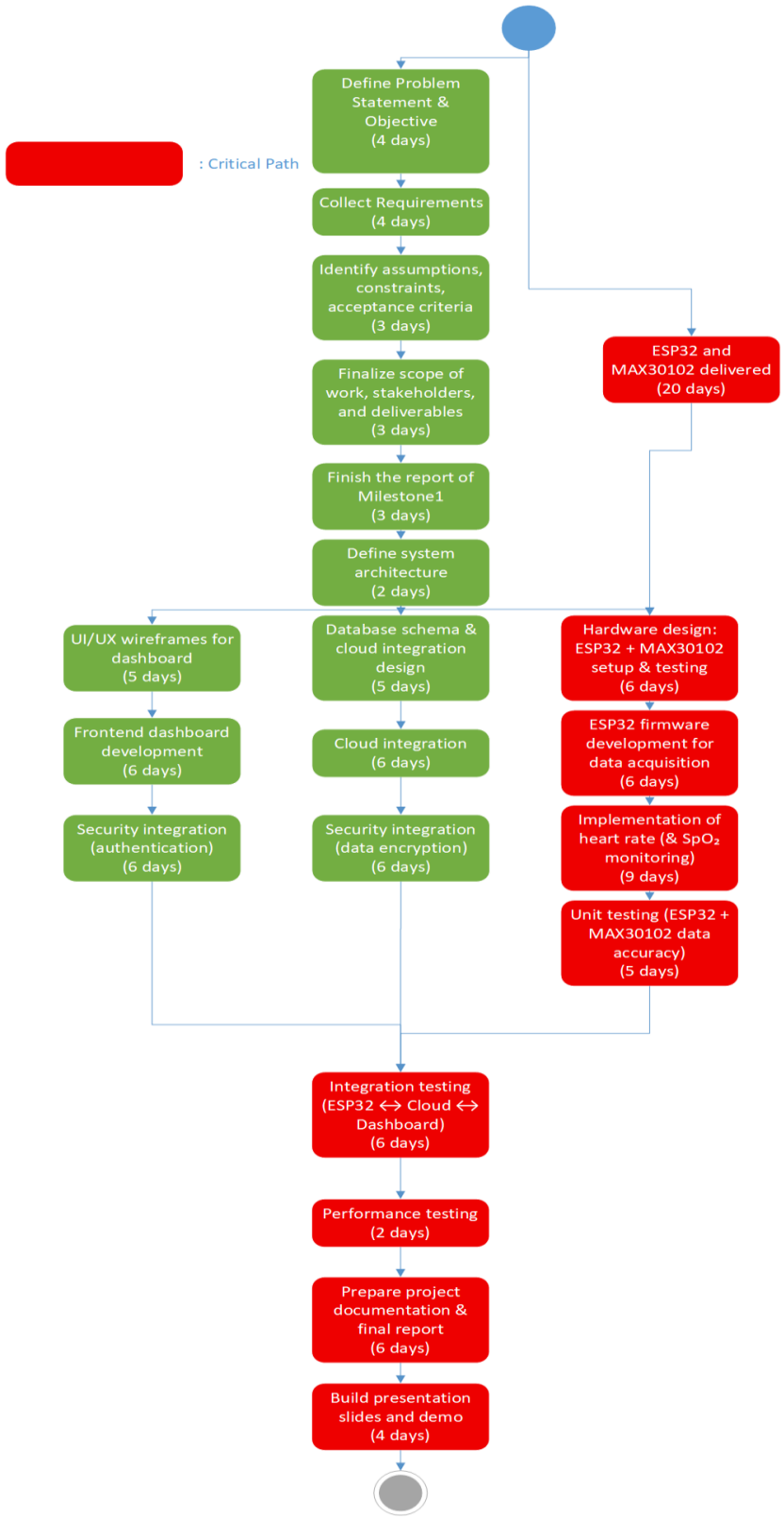

Figure 2: Critical Path View

2.Problem Statement

In today's fast-paced world, the risk of heart-related issues is increasing due to stress, unhealthy lifestyles, and pre-existing health conditions. Consequently, individuals are becoming more conscious of their cardiovascular health and the need for regular monitoring.

This project aims to address this problem by developing a low-cost, IoT-based health monitoring system using the MAX30102 sensor and ESP32 microcontroller. The system will measure heart rate and blood oxygen levels, transmit the data to a cloud platform, and display it through a user-friendly web dashboard, enabling users to monitor their vital signs in real time and gain insights into their cardiovascular health.

3.Requirements Gathering

The requirement's gathering process is a critical step in any project, as it defines what the system is expected to achieve. This phase can be divided into three main categories: Functional Requirements, which specify how the system should operate; Non-Functional Requirements, which define the performance and quality attributes of the system; and Assumptions, Constraints, and Acceptance Criteria, which establish project boundaries, implementation standards, and the conditions for project success.

3.1 Functional Requirements

3.1.1 Sensor Data Collection:

- The system must measure heart rate (BPM) using the MAX30102 sensor.
- The system may measure blood oxygen level (SpO_2) using the MAX30102 sensor.

3.1.2 Data Processing and Transmission:

- The ESP32 must process raw signals from the MAX30102 into usable metrics.
- The ESP32 must transmit processed data to a cloud server or web application via Wi-Fi or Bluetooth.

3.1.3 Data Storage:

- The system must store collected data in a cloud database for future reference and trend analysis.

3.1.4 System Control:

- Users may be able to start and stop monitoring via the web interface.

3.1.5 Prototype Testing & Validation:

- The system must be tested with real human readings to verify accuracy.
- The prototype must undergo connectivity testing to ensure reliable Wi-Fi or Bluetooth transmission.

3.2 Non-Functional Requirements

3.2.1 Performance

- The system must capture and process heart rate (and SpO_2) data in real time with a maximum latency of ≤ 2 seconds between sensor reading and dashboard display.

3.2.2 Reliability & Availability

- The system must provide at least 95% uptime during testing and demonstrations.

3.2.3 Scalability

- The cloud platform must allow for at least 5 simultaneous users viewing the dashboard without performance degradation.

- The database must be capable of storing at least 1 month of continuous data per user.

3.2.4 Usability

- The web interface must be friendly and accessible on standard browsers.
- The dashboard must use clear graphs and color indicators (e.g., green for normal, red for abnormal readings) to ensure easy interpretation.

3.2.5 Maintainability and Extensibility

- The system must be modular, allowing easy migration to another cloud provider without major code changes.
- Documentation must be provided for hardware setup, software code, and cloud configuration to support future improvements.

3.2.6 Security & Privacy

- Data transmission between ESP32 and cloud must be encrypted using HTTPS or equivalent.
- User data must be accessible only with secure login credentials.

3.2.7 Compliance (Ethical & Safety)

- The system must include a disclaimer that it is for educational and demonstration purposes only, not for clinical diagnosis.
- The device must operate within safe limits of LED light exposure defined by the MAX30102 datasheet.

3.3 Assumptions, Constraints, and Acceptance Criteria

3.3.1 Assumptions

- **User Environment:**
  - Users will remain relatively still during measurements to minimize motion artifacts.
  - The finger placement on the MAX30102 sensor will be correct and consistent.
- **Hardware Availability:**
  - The ESP32 board and MAX30102 sensor module are available and functional throughout the project.
  - A stable power source (USB or battery) is available for continuous operation.
- **Connectivity:**
  - A stable Wi-Fi network is available for data transmission from ESP32 to the cloud.
  - Internet access will be available for cloud database and dashboard hosting
- **Software & Tools:**
  - Open-source libraries for ESP32–MAX30102 communication and data processing are available and reliable.
  - Cloud services will be accessible within free or academic tiers.
- **Project Scope & Constraints:**

- The project will focus on heart rate (and blood oxygen) only; no other medical parameters will be included.
- The system will be used strictly for educational and demonstration purposes, not for clinical use.

3.3.2 Constraints

- **Hardware Limitations:**
  - The accuracy of heart rate and SpO_2 readings is limited by the MAX30102 hardware capabilities and may not match medical-grade devices.
- **Budget:**
  - The project budget is limited to low-cost components (< $100 CAD), restricting the choice of sensors, cloud services, and development tools.
- **Technical Constraints:**
  - The ESP32 has limited processing power and memory, restricting the complexity of onboard data analysis.
  - Data analysis beyond basic processing (e.g., advanced AI/ML models) must be handled in the cloud, not on the ESP32.
- **Data & Connectivity:**
  - Reliable Wi-Fi connectivity is required; performance may degrade in unstable network environments.
- **Ethical & Safety Constraints**
  - The system must clearly state it is for educational/demonstration purposes only and not intended for clinical diagnosis or treatment.
  - Testing will be conducted on healthy volunteers only, not on patients with medical conditions.

4.Web Search

Following comprehensive research and analysis, our team identified the most suitable technologies and methodologies for developing the capstone project system. The objective is to ensure that the solution is reliable, efficient, and user-friendly while maintaining scalability and cost-effectiveness. The selected approaches are designed to optimize system performance, enhance data security, and improve the overall user experience.

4.1 Hardware Development Technologies and Frameworks

Our project relies on the ESP32 microcontroller and the MAX30102 pulse oximeter/heart rate sensor. Together, they form the core hardware layer for real-time physiological data acquisition, processing, and communication. The hardware development is supported by microcontroller programming frameworks, peripheral libraries, and testing tools to ensure accurate data collection and efficient device operation

4.1.1 Sensor Integration Technologies

- Arduino Core for ESP32: The Arduino Core provides a simplified development environment for programming the ESP32. It offers high-level APIs for GPIO, I²C, SPI, WiFi, and sensor integration. We chose Arduino Core because it speeds up development through pre-built libraries, including support for the MAX30102 sensor, and integrates easily with the Arduino IDE or PlatformIO. In our project, Arduino Core is used for rapid prototyping, handling sensor data acquisition, and transmitting results via WiFi or Bluetooth.
- ESP-IDF (Espressif IoT Development Framework): ESP-IDF is the official framework from Espressif, offering low-level hardware control, multitasking via FreeRTOS, and optimized networking stacks. While it is more complex than Arduino Core, ESP-IDF provides greater flexibility and fine-grained resource management. In the current phase, we will not rely on ESP-IDF to avoid overcomplicating development. However, if additional time is available in later project stages, we will consider migrating selected modules (e.g., real-time multitasking or advanced power optimization) to ESP-IDF to enhance system robustness.

4.1.2 Sensor Integration Technologies

- I²C Communication Protocol: The MAX30102 communicates with the ESP32 through the I²C protocol. We chose I²C because it is efficient, requires only two data lines (SDA and SCL), and supports reliable communication between microcontrollers and low-power sensors. In our project, I²C is used to transmit raw infrared and red-light readings from the MAX30102 to the ESP32 for further processing.
- MAX30102 Sensor Library: Open-source MAX30102 libraries are used to simplify the interaction with the sensor. These libraries provide functions for initializing the sensor,

reading raw photoplethysmography (PPG) signals, and calculating heart rate and SpO_2 levels. Using these libraries allows us to focus on algorithm implementation instead of low-level sensor configuration.

4.1.3 Development Tools

- Arduino IDE / PlatformIO: Arduino IDE is used for quick development and deployment, while PlatformIO (inside Visual Studio Code) is used for more advanced debugging and dependency management. Both tools allow uploading code to the ESP32, monitoring serial output, and testing sensor data in real time.
- Serial Monitor: For debugging I^2C signals and verifying communication between ESP32 and MAX30102, the built-in Arduino Serial Monitor is used. This helps ensure the reliability of data transfer and validates sensor initialization.

4.1.4 Wireless Communication Frameworks

- WiFi (IEEE 802.11 b/g/n): WiFi is integrated into the ESP32 and is used to transmit health data to a remote server or web application. We selected WiFi because it provides high data throughput and supports direct integration with cloud platforms for real-time monitoring.
- Serial Monitor: For debugging I^2C signals and verifying communication between ESP32 and MAX30102, the built-in Arduino Serial Monitor is used. This helps ensure the reliability of data transfer and validates sensor initialization.

4.1.5 Algorithmic Processing

- Signal Processing Libraries (C/C++): The raw data from the MAX30102 requires filtering (e.g., moving average or FIR filters) and peak detection algorithms to calculate accurate heart rate and SpO_2. C/C++ libraries for digital signal processing are integrated within the ESP32 codebase to ensure real-time performance.

4.2 Maintenance and Version Control

To ensure stability, reliability, and long-term usability of the project, we will implement clear maintenance procedures and a structured version control strategy.

4.2.1 Maintenance

- Bug Fixes and Updates: Any errors identified during development or operation will be documented, fixed, and validated through a structured process to maintain system reliability.
- Dependency Management: Regular updates of ESP32 SDK, Arduino Core, or ESP-IDF libraries will be performed to minimize security risks and prevent outdated dependencies.
- Hardware Maintenance: The ESP32 development board and MAX30102 sensor will be periodically tested to ensure stable performance. Damaged modules will be replaced promptly.

- Documentation Updates: Technical documentation, including system architecture, interface specifications, and user guides, will be continuously updated to support both current and future team members.

4.2.2 Version Control

- Git Version Control System: The project source code will be hosted on GitHub to ensure transparency, collaboration, and traceability of changes.
- Code Review Process: All contributions will undergo peer review before merging into the main branch to ensure code quality, maintainability, and consistency.

4.3 Frontend Technologies

HeartLink's web dashboard integrates frontend web technologies, cloud platforms, and API connections to visualize real-time biomedical data. The dashboard serves as the main interface where users can view live readings, trends, and alerts received from the ESP32 through cloud synchronization.

4.3.1 HTML5, CSS3, and JavaScript

- Structure and style dashboard components.
- Implement interactive features such as real-time charts and alerts.
- Ensure responsiveness and cross-browser compatibility.

4.3.2 React.js (Frontend Framework)

- Display real-time heart rate and PPG data from the cloud.
- Manage UI components such as graphs, status indicators, and alert panels.
- Handle state updates efficiently for continuous data refresh.

4.3.3 Looker Studio (Visualization Libraries)

- Display real-time heart rate data trends.
- Visualize pulse variations and time-based patterns.
- Offer clear and engaging data presentation for users.

4.3.4 API and WebSocket Communication

- Instant updates of heart rate data.
- Reliable synchronization with minimal delay.
- Continuous data flow for live visualization.

4.4 User Interface (UI) and User Experience (UX) Design

HeartLink's dashboard design emphasizes clarity, simplicity, and accessibility for users viewing health data in educational and non-clinical contexts.

4.4.1 UI/UX principles

- Clean layout with clearly labeled sections for heart rate, signal strength, and system status.

- Use of color coding (e.g., green for normal, red for high readings).
- Responsive design for accessibility on laptops, tablets, and desktops.
- Easy navigation with intuitive menus and icons.
- Consistent typography and contrast for readability.

4.4.2 Usability features

- Clean layout with clearly labeled sections for heart rate, signal strength, and system status.
- Use of color coding (e.g., green for normal, red for high readings).
- Responsive design for accessibility on laptops, tablets, and desktops.
- Easy navigation with intuitive menus and icons.
- Consistent typography and contrast for readability.

4.5 Data Engineering Pipeline

The data engineering pipeline for this project involves several key stages: Data Acquisition (from the sensor), Ingestion (transferring data to the cloud), Storage & Processing, and finally Visualization.

4.5.1 Data Ingestion (Sensor to Cloud)

- Google Cloud Pub/sub (Real-time Data Ingestion): Pub/Sub is a fully managed, serverless, real-time messaging service. It acts as a buffer, reliably ingesting high-volume, continuous streams of data from your ESP32/Raspberry Pi. It decouples the data sender (sensor) from the data receiver (database/pipeline), ensuring that even if your device sends bursts of data, no data is lost during pipeline processing.
- Google Cloud IoT Core (Cloud-Side Authentication): IoT Core provides secure, direct connections for global devices. It handles device authentication (crucial for capstone security), connection, and communication, pushing all data directly into a Pub/Sub topic.

4.5.2 Data Storage and Processing

- Google BigQuery (Cloud Data Warehouse): BigQuery is a highly scalable, serverless data warehouse that is optimized for fast, complex SQL queries on massive datasets. Crucially, it offers native, high-performance integration with Looker Studio (formerly Data Google Studio), making it the ideal final storage layer before visualization. It handles the high volume and velocity of time-series sensor data perfectly.
- Google Cloud Dataflow or BigQuery SQL (Data Transformation): BigQuery is a highly scalable, serverless data warehouse that is optimized for fast, complex SQL queries on massive datasets. Crucially, it offers native, high-performance integration with Looker Studio (formerly Data Google Studio), making it the ideal final storage layer before visualization. It handles the high volume and velocity of time-series sensor data perfectly.

4.5.3 Visualization and Reporting

- Google Looker Studio (Automation Dashboard): As requested, this tool is free to use and provides an intuitive, web-based interface for creating interactive, automated dashboards. It has a native, high-speed connector to BigQuery, allowing for real-time visualization of the ingested heart pulse and $¥text{SpO}_2$ data without needing to move or copy data.

5.Data Dictionary

5.1 Hardware Layer Data Dictionary

5.1.1 Physical/I²C & GPIO configuration

Varibles	Type	Unit/Format	Description
i2cPort	enum (esp-idf)	-	I²C controller used for MAX30102.
i2cSdaPin	gpio_num_t	-	SDA GPIO for MAX30102.
i2cSclPin	gpio_num_t	-	SCL GPIO for MAX30102.
i2cMasterFreqHz	uint32_t	Hz	I²C bus frequency.
max30102Addr	uint8_t	7-bit addr	I²C address of MAX30102.
max30102IntPin	gpio_num_t	-	GPIO for MAX30102 active-low interrupt.

Table 2: Data Dictionary of Hardware Layer (Physical/I²C & GPIO configuration)

5.1.2 Sensor operating mode & LED drive

Varibles	Type	Unit/Format	Description
mode	sensorMode	-	Current measurement mode.
ledCurrentRed	uint8_t	~mA per datasheet	Red LED current register (REG_LED2_PA).

ledCurrentIr	uint8_t	~mA per datasheet	IR LED current register (REG_LED1_PA).
fifoConfig	uint8_t	bitfield	FIFO config written during init.

Table 3: Data Dictionary of Hardware Layer (Sensor operating mode & LED drive)

5.1.3 Sampling & pulse width enums

Varibles	Type	Unit/Format	Description
sampleRate	max30102SampleRate	sps	SpO_2/PPG sample rate selection.
pulseWidth	max30102PulseWidth	µs	LED pulse width / ADC resolution.

Table 4: Data Dictionary of Hardware Layer (Sampling & pulse width enums)

5.1.4 Interrupt status & masks

Varibles	Type	Unit/Format	Description
intStatus1	uint8_t	bitmask	INT_STATUS_1 (read-to-clear).
intStatus2	uint8_t	bitmask	INT_STATUS_2 (read-to-clear).
intMaskDataReady	const uint8_t	-	Data-ready flag in INT_STATUS_1.
intMaskFifoAlmostFull	const uint8_t	-	FIFO almost full.
intMaskAlcOvf	const uint8_t	-	Ambient light overflow.

intMaskProximity	const uint8_t	-	Proximity threshold event.

Table 5: Data Dictionary of Hardware Layer (Interrupt status & masks)

5.1.5 Sample payload

Varibles	Type	Unit/Format	Description
red	uint32_t	raw ADC counts	18-bit RED PPG sample assembled from FIFO
ir	uint32_t	raw ADC counts	18-bit IR PPG sample assembled from FIFO.

Table 6: Data Dictionary of Hardware Layer (Sample payload)

5.2 Front-End Layer Data Dictionary

5.2.1 User Interface Data

Varibles	Type	Description
screen_name	string	Identifies the current view (e.g., Dashboard, Settings, History).
deviceId	enum	Determines layout appearance (light/dark modes).
redAdc	int	Tracks the active UI route.

Table 7: Data Dictionary of User Interface

5.2.2 Vital Signs Data

Varibles	Type	Description
heart_rate	string	Representing BPM (beats per minute).

spo2_level (optional)	float	Blood oxygen saturation percentage.
timestamp	string/date time	The time of recording.

Table 8: Data Dictionary of Vital Signs Data

5.2.3 Device and Connectivity Data

Varibles	Type	Description
device_id	string	Unique identifier for the ESP32 unit.
connection_status	enum	Indicates the strength of the signal.
last_sync_time	string/date time	The time of the last synchronous.

Table 9: Data Dictionary of Device and Connectivity Data

5.2.4 Alert and Notification Data

Varibles	Type	Description
alert_message	string	Display the alert messages to users.

Table 10: Data Dictionary of Hardware Laye

5.2.5 User and Settings Data

Varibles	Type	Description
user_id	string	Unique identifier for the user.

preferred_units	enum	Choose the display unit (e.g. BPM).
notifications_enabled	bool	Turn on/off the notifications.
cloud_sync_enabled	bool	Boolean flags synchronized to the cloud.
Language/locale	string	String for regional formatting.
theme_preference	enum	Choose the theme (e.g. Light, Dark, System).

Table 11: Data Dictionary of User and Settings Data

5.2.6 Historical and Trend Data

Varibles	Type	Description
history_records	Arrays of heart_rate, spo2_level and timestamp	Arrays of heart_rate, spo2_level and timestamp.
date_range_filter	string	The range of date.
chart_data	Arrays for graph plotting	Arrays for graph plotting.

Table 12: Data Dictionary of Historical and Trend Data

5.3 Cloud Layer Data Dictionary

5.3.1 User Entity

Attribute Name	Data Type	Key Type	Description	note
User_ID	STRING	PK	Unique identifier for any actor	Assigned UUID.
First_Name	STRING		User's first name.	Required.
Last_Name	STRING		User's last name.	Required.
User_Role	STRING		The actor's role in the system.	Enum : "Admin".
Email	STRING	Unique	User's primary contact and login.	Must be unique.
Phone_Number	STRING		User's mobile number for notifications.	

Table 13: Data Dictionary of User Entity

5.3.2 Device Entity

Attribute Name	Data Type	Key Type	Description	note
Device_ID	STRING	PK	Unique identifier for the physical IoT device.	Unique per hardware unit.
Patient_User_ID	STRING	FK	The ID of the patient to whom the device is assigned.	Links to USER.User_ID.
Model	STRING		Hardware model of the monitoring unit.	e.g., "ESP32/MAX30102".
Registration_Date	TIMESTAMP		The date the device was first activated.	

Table 14: Data Dictionary of Device Entity

5.3.3 Reading Entity (time-series Data)

Attribute Name	Data Type	Key Type	Description	note

Reading_ID	STRING	PK	Unique identifier	Generated UUID.
Device_ID	STRING	FK	The device that generated this measurement.	Links to DEVICE.Device_ID.
Reading_Timestamp	TIMESTAMP	PK	The exact time the measurement was taken (UTC).	Crucial for ordering and partition key.
Heart_Rate_BPM	INTEGER		The calculated heart rate in beats per minute.	Expected range: 30 - 250.
SpO2_Level	FLOAT		The calculated blood oxygen saturation level.	Expected range: 80.0 - 100.0.

Table 15: Data Dictionary of Reading Entity

5.3.4 Threshold Entity

Attribute Name	Data Type	Key Type	Description	note
Threshold_ID	STRING	PK	Unique identifier for this set of patient thresholds.	
Patient_User_ID	STRING	PK, FK	The patient this threshold applies to.	Links to USER.User_ID.
Min_Heart_Rate	INTEGER		Minimum acceptable Heart Rate (BPM).	
Max_Heart_Rate	INTEGER		Maximum acceptable Heart Rate (BPM).	
Min_SpO2	FLOAT		Minimum acceptable SpO2 level.	
Set_By_User_ID	STRING	FK	The Doctor/User who configured this threshold set.	Links to USER.User_ID.

Table 16: Data Dictionary of Threshold Entity

5.3.5 Alert Entity (Event Data)

Attribute Name	Data Type	Key Type	Description	note
Alert_ID	STRING	PK	Unique identifier for a triggered alert instance.	
Reading_ID	STRING	FK	The specific reading that triggered this alert.	Links to READING.Reading_ID.
Patient_User_ID	STRING	FK	The patient who triggered the alert.	Links to USER.User_ID.
Alert_Timestamp	TIMESTAMP		The time the system recognized the alert condition.	
Type	STRING		The condition that caused the alert.	Enum: "HR_High", "HR_Low", "SpO2_Low".
Is_Acknowledged	BOOLEAN		Flag if a professional has closed the alert.	Default: FALSE.
Acknowledging_User_ID	STRING	FK	The ID of the User who closed the alert.	Links to USER.User_ID. (Nullable if not acknowledged).

Table 17: Data Dictionary of Alert Entity

6.Embedded Services and Third-Party Integrations

6.1 ESP-IDF

Figure 3: ESP-IDF

ESP-IDF serves as the foundation for the embedded layer, enabling the ESP32 to collect sensor data from the MAX30102 through I²C communication, process the signals in real time, and securely transmit the results to the cloud over Wi-Fi (HTTPS/TLS). It also integrates FreeRTOS, allowing efficient multitasking and real-time handling of asynchronous sensor readings and network operations.

6.2 Google Cloud Pub/Sub

Figure 4: Google Cloud Pub/Sub

Google Cloud Pub/Sub streams our incoming sensor data, providing reliable, scalable message queuing between devices and cloud processing pipeline.

6.3 Google Cloud Storage & Google Big Query

Figure 5: Google Cloud Storage

Figure 6: Google Big Query

Use Cloud Storage for raw data (files), and BigQuery for structured, time-stamped heart rate entries. BigQuery is ideal for querying large datasets efficiently and powers downstream analytics.

6.4 Firebase Authentication

Figure 7: Firebase

Firebase Authentication creates secure user accounts, protects access to data, and supports privacy requirements for health data access.

6.5 Looker Studio

Figure 8: Looker Studio

Looker Studio builds interactive dashboards, displaying heart rate trends, historical reports, and real-time alerts for users.

6.6 Google Cloud Run

Figure 9: Google Cloud Run

Google Cloud Run help to providing a secure, scalable, and flexible platform to deploy our web-based dashboard, API, and backend data processing.

7.UI and UX Prototypes

7.1 Hardware Prototype

Figure 10: Heartlink

7.2 Front-end Prototype

https://www.figma.com/make/rU63BB8kvgExZDKW5ixYQa/Wearable-Health-Monitor?node-id=0-1&t=g6IvCl59lmOZ5bmX-1

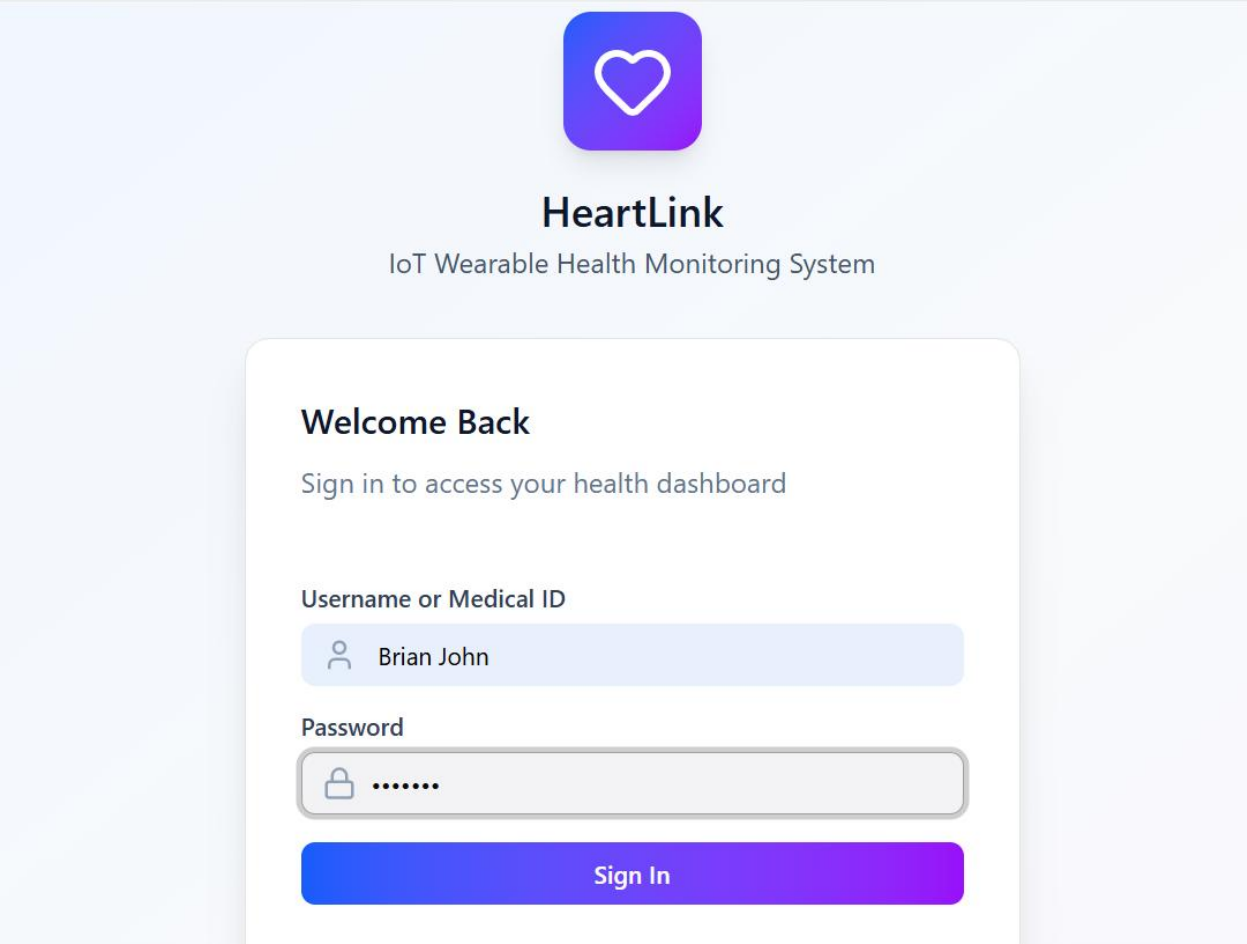

Figure 11: Login

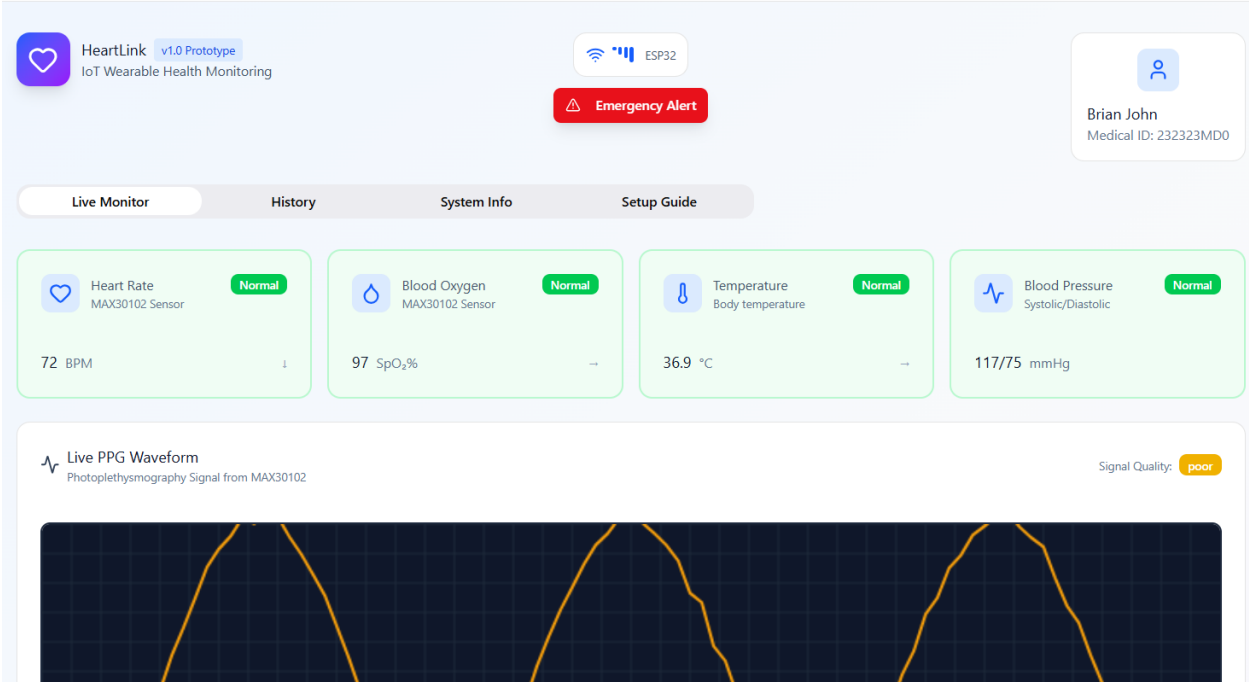

Figure 12: User Dashboard

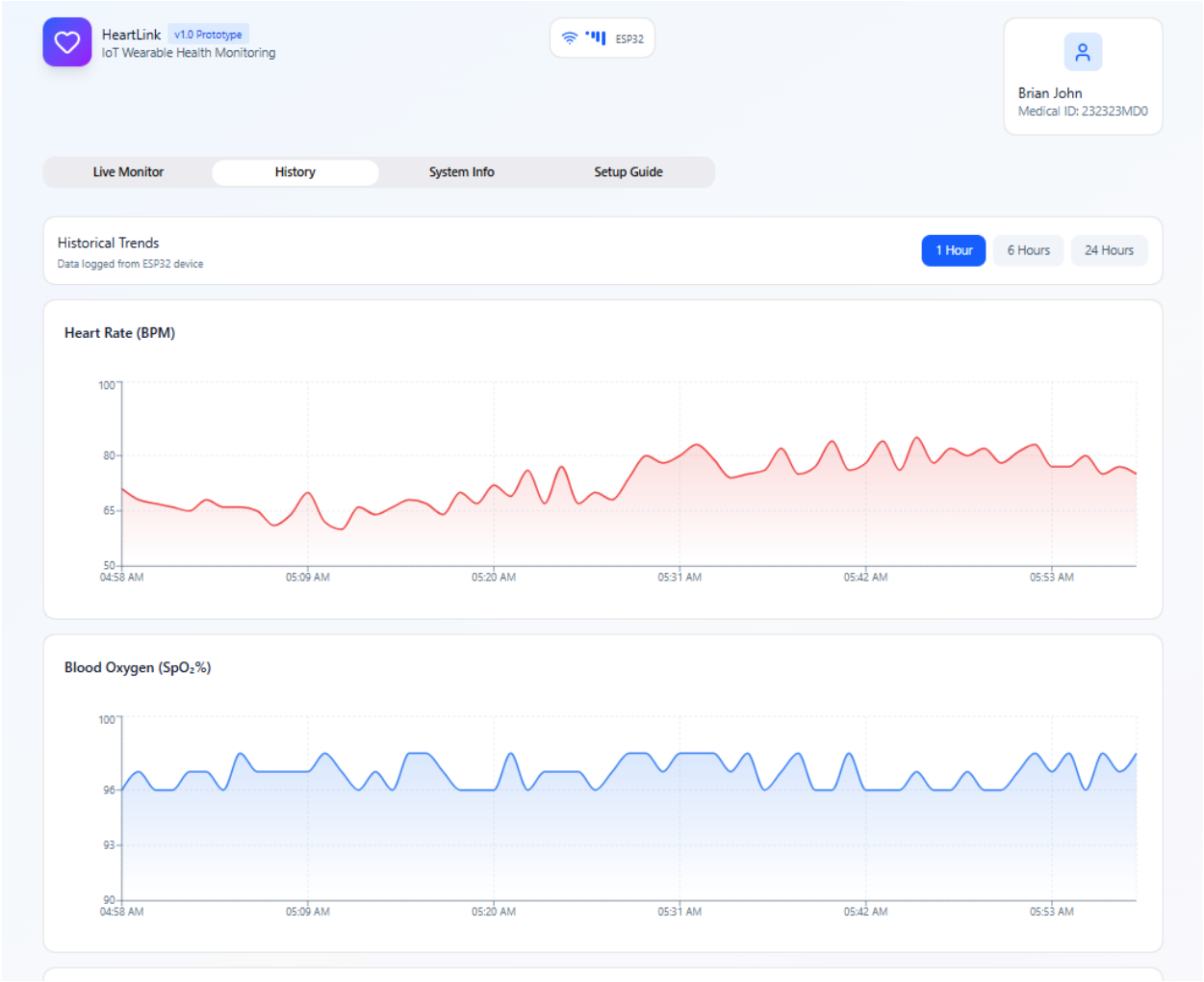

Figure 13: History

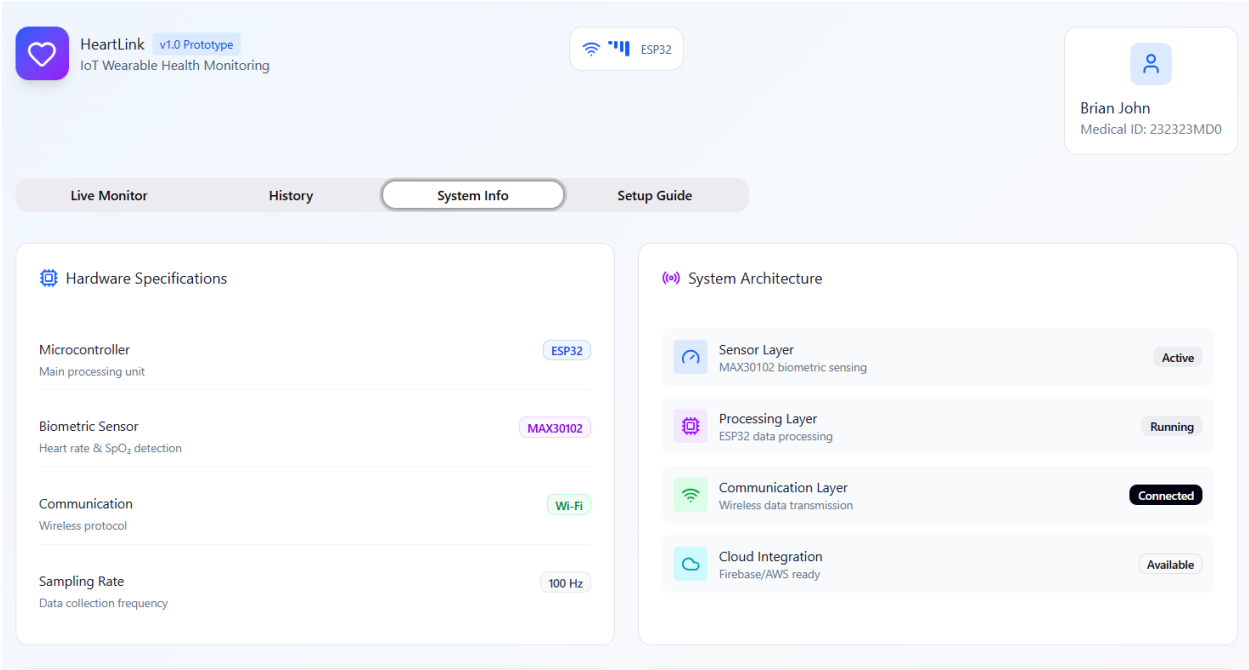

Figure 14: System Info

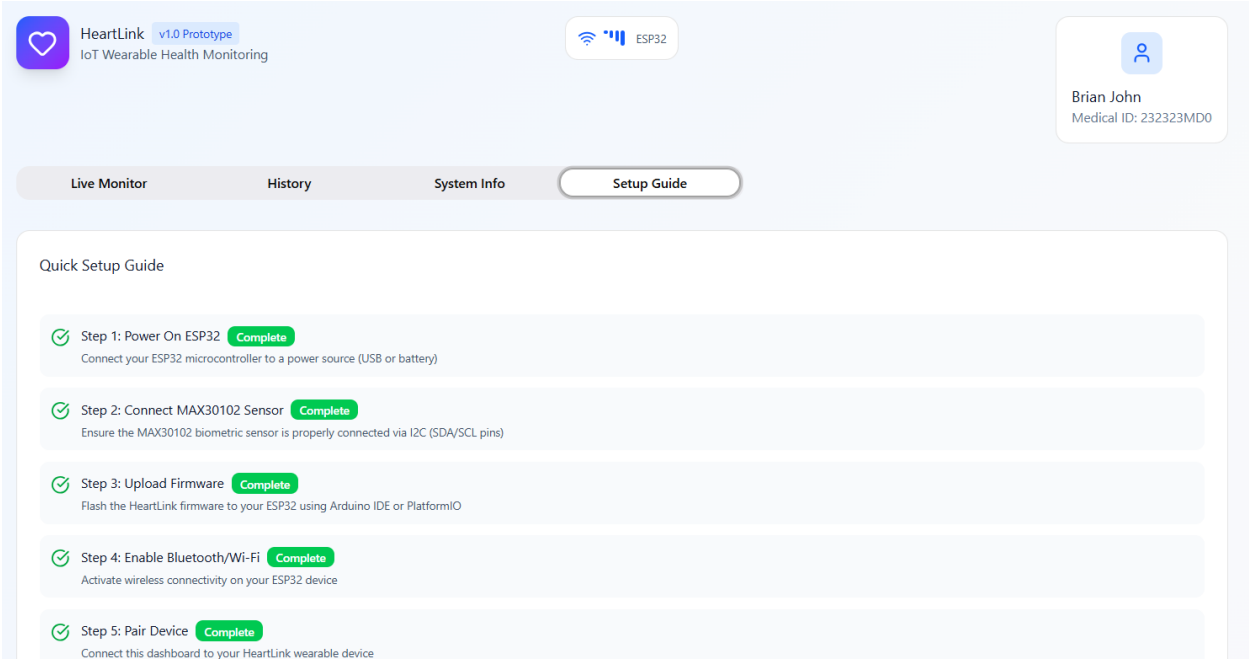

Figure 15: Set Up Guide

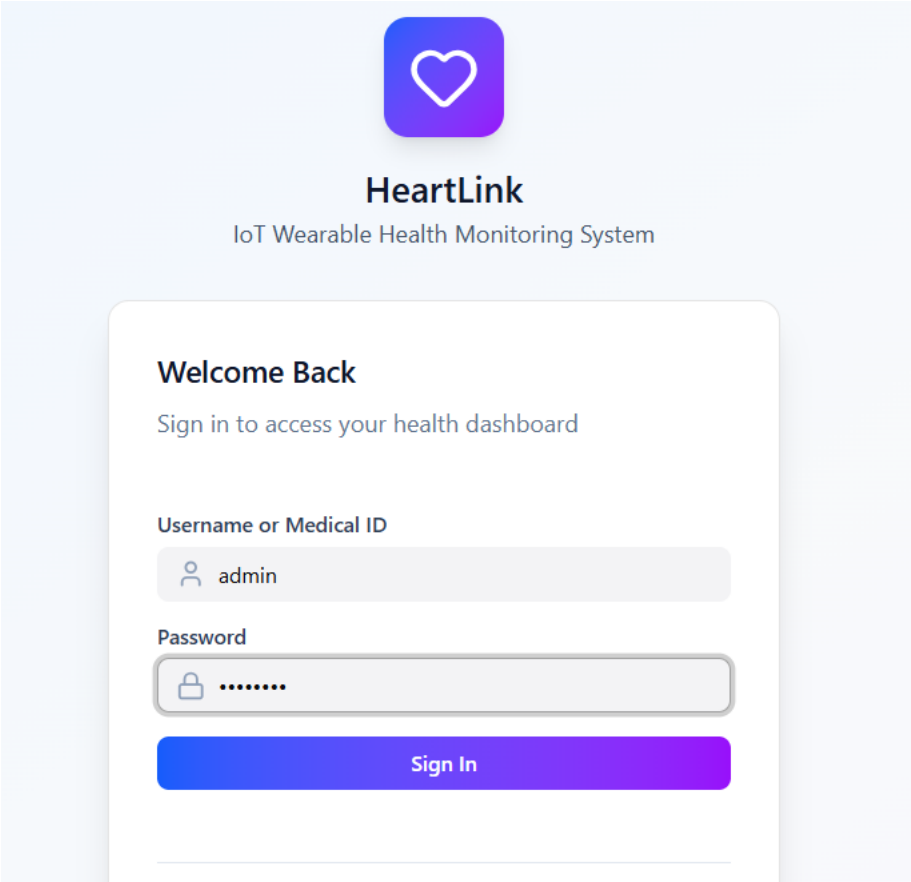

Figure 16: Admin Login

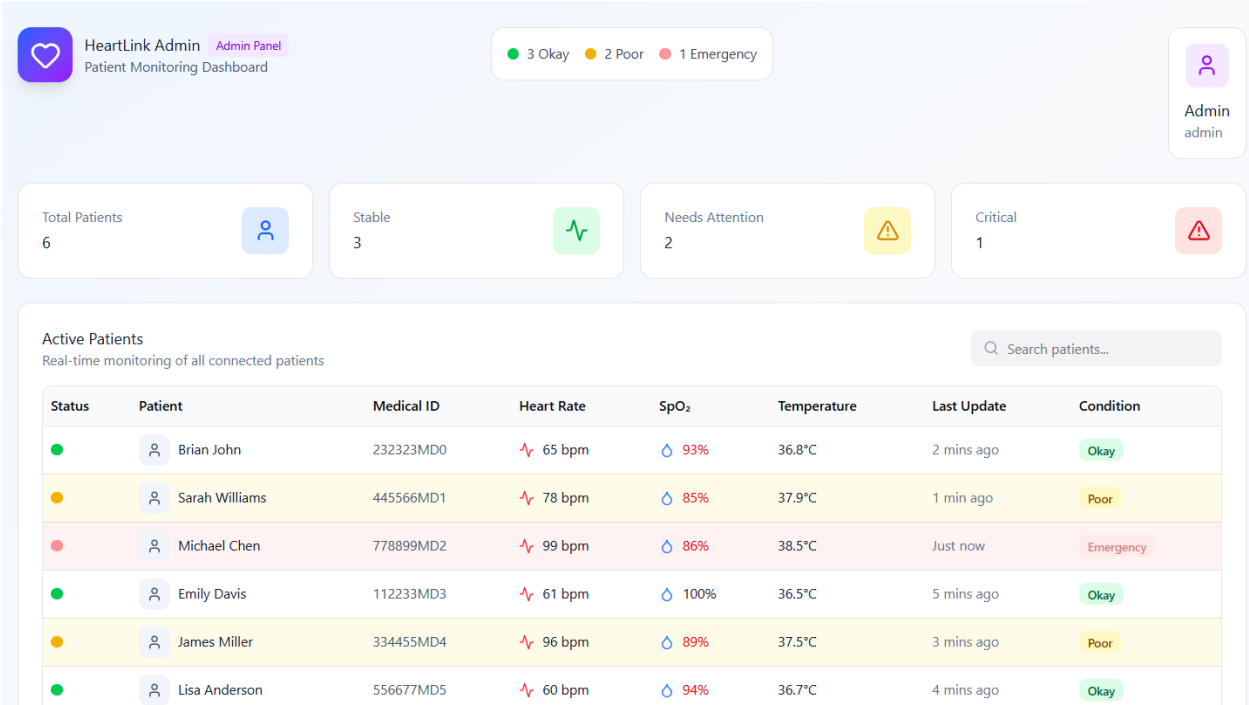

Figure 17: Admin Dashboard

8.Project Design

8.1 System Architecture

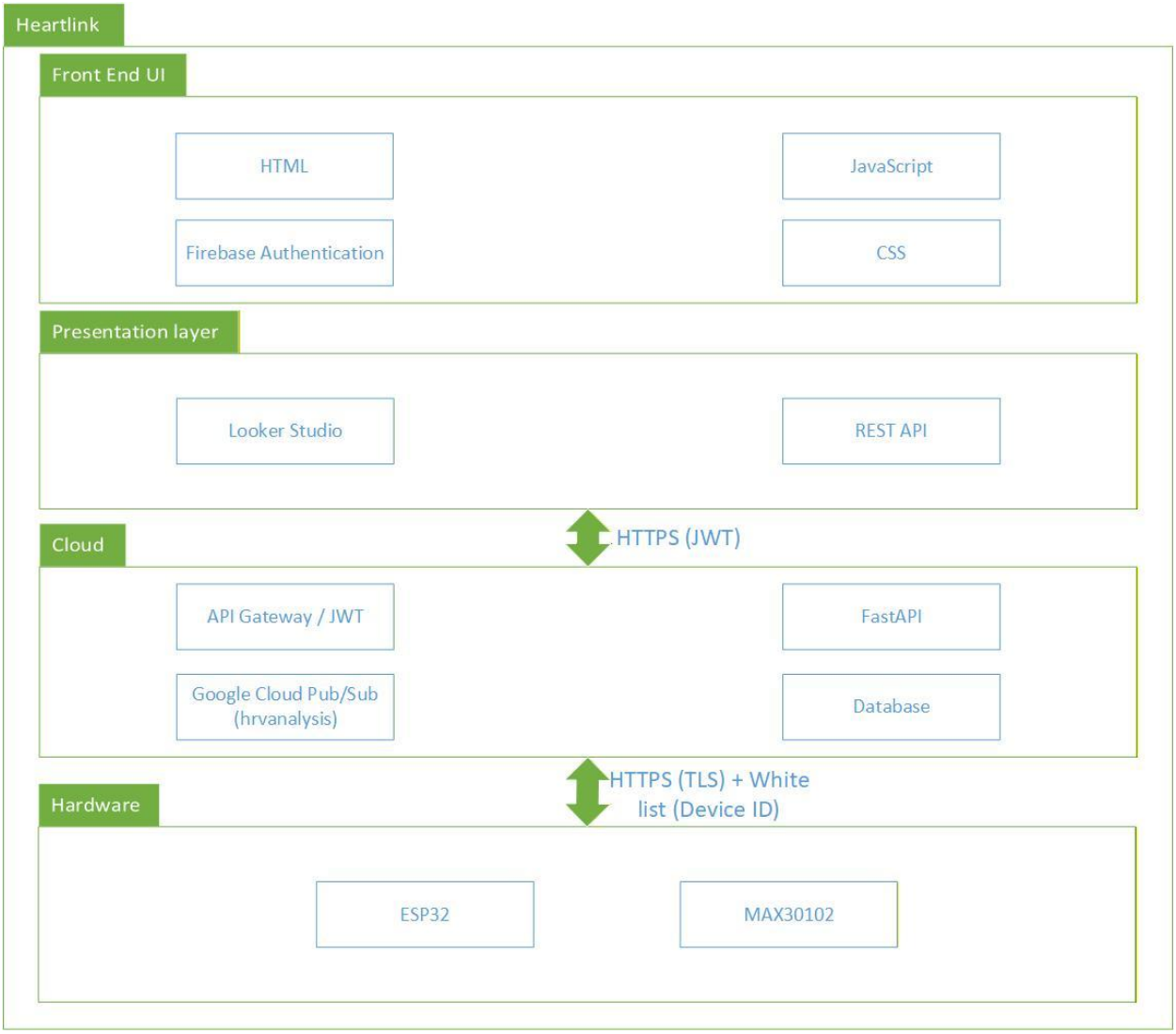

Figure 18: System Architecture

The Heartlink system is built as a secure IoT-cloud health monitoring platform.

- At the Hardware layer, the MAX30102 sensor captures photoplethysmography (PPG) signals, which are processed by the ESP32 microcontroller using the ESP-IDF framework. The ESP32 securely sends data to the cloud via HTTPS (TLS) with device whitelisting for authentication.
- In the Cloud layer, data flows through the API Gateway using JWT-based authentication before being handled by FastAPI services. Processed data and results are published via Google Cloud Pub/Sub, stored in the database, and used for HRV analysis.
- The Presentation layer provides data visualization through Looker Studio and a REST API, offering analytics dashboards and enabling the front-end to access processed health insights.

Finally, the Front-End UI—built with HTML, CSS, and JavaScript—integrates Firebase Authentication for secure user access, allowing users to view personalized results and interact with the system through an intuitive web interface.

8.2 Hardware UML Diagram

8.2.1 Flow Chart

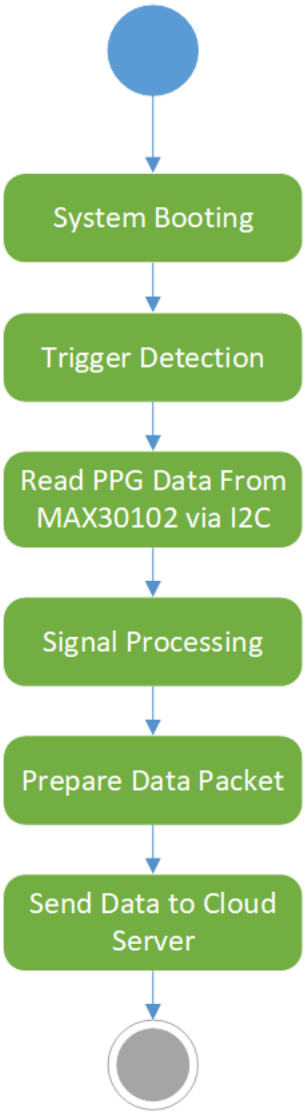

Figure 19: Flow Chart

8.2.2 Sequence Diagram

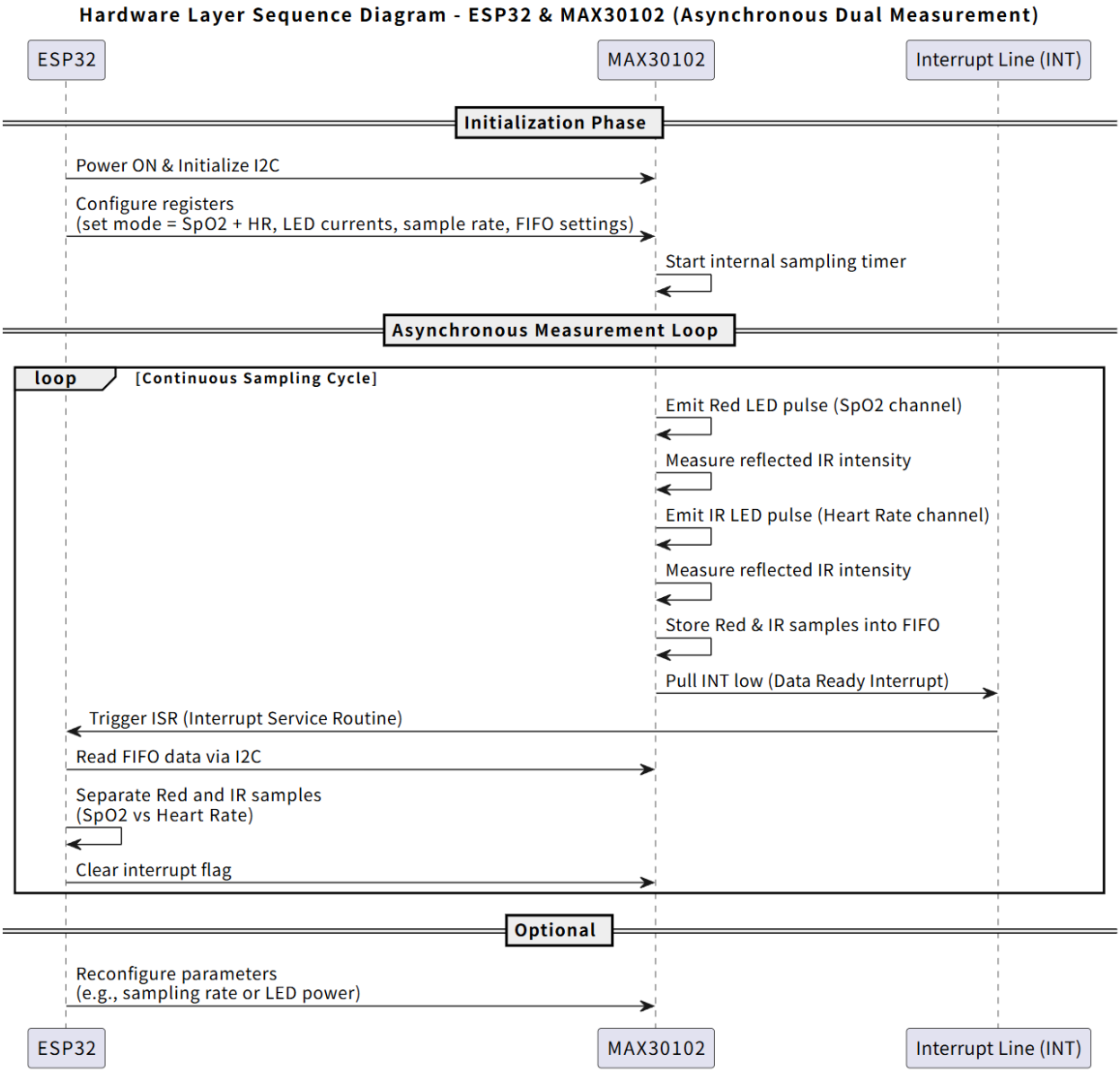

Figure 20: Sequence Diagram

8.3 Front-End UML Diagrams

8.3.1 Use Case Diagram

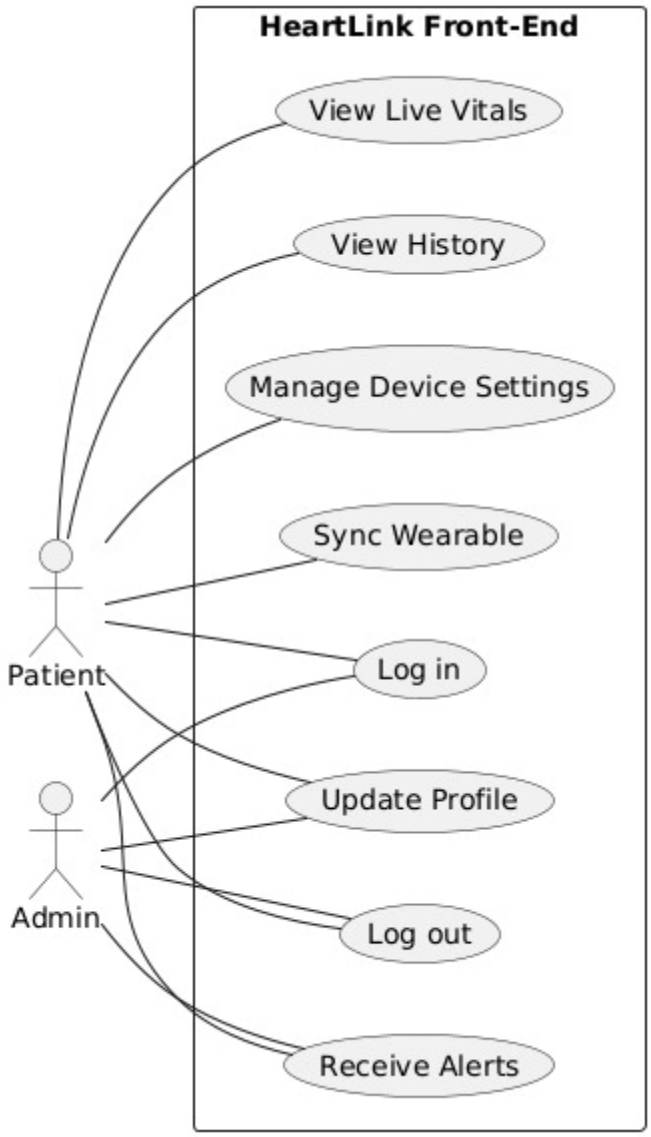

Figure 21: Use Case Diagram

8.3.2 Activity Diagram

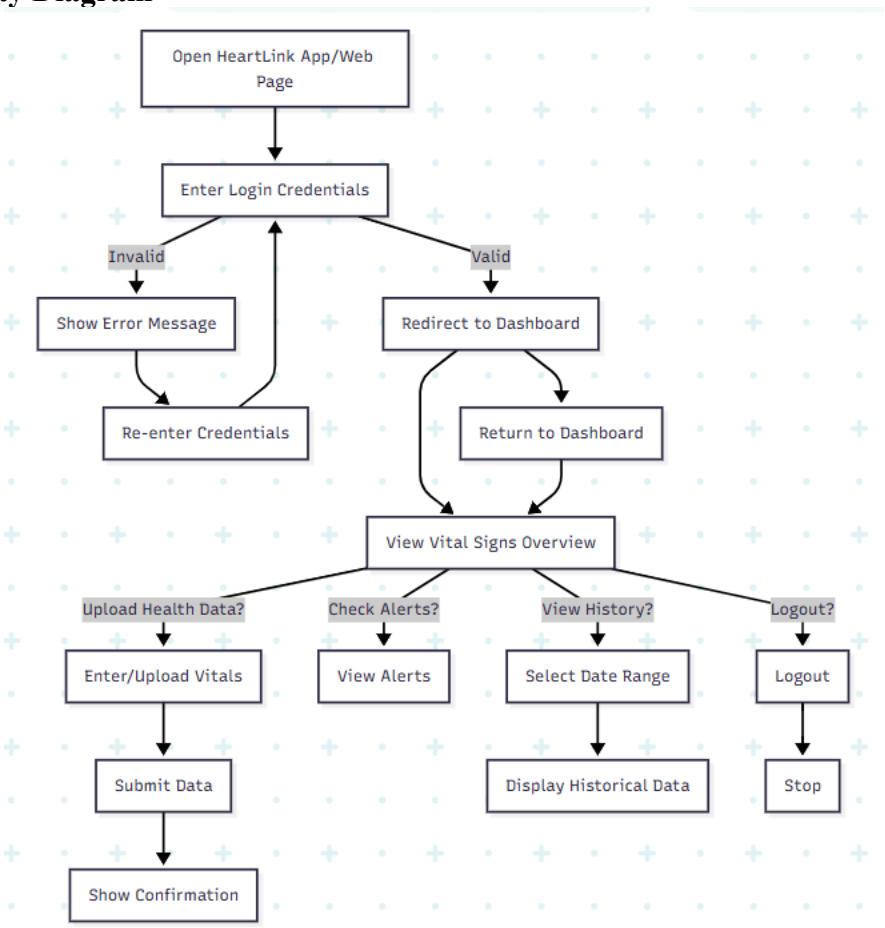

Figure 22: Activity Diagram

8.4 Cloud Layer UML Diagrams

8.4.1 Class Diagram

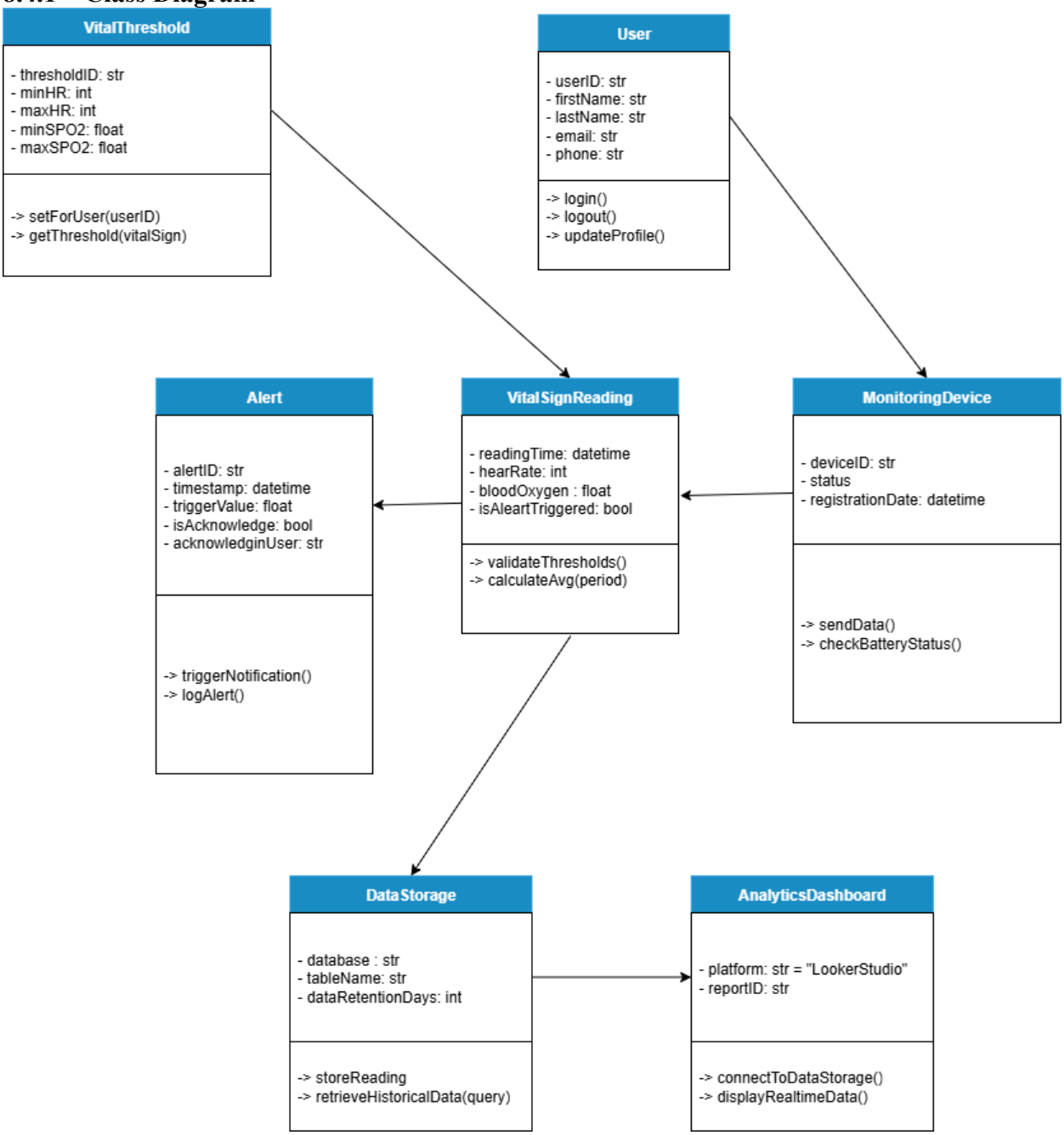

Figure 23: Class Diagram

8.4.2 Flow Chart

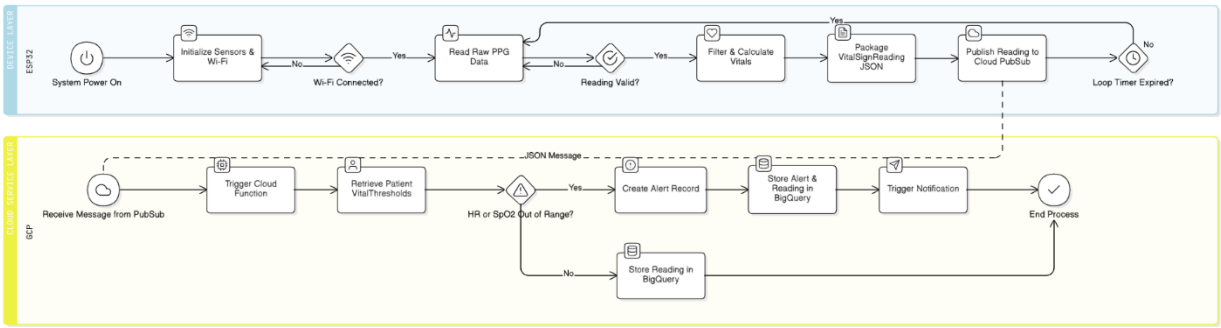

Figure 24: Flow Chart

8.5 Entity Relationship Diagram

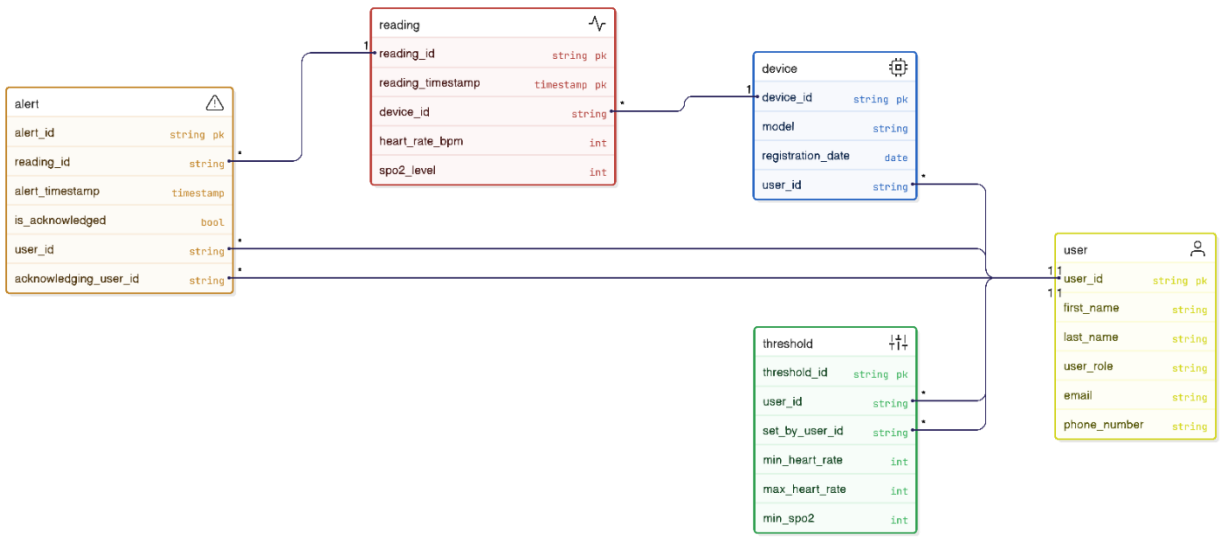

Figure 25: Entity Relationship Diagram

The Heartlink database consists of five main entities: User, Device, Reading, Threshold, and Alert. Each device is registered to a user and records heart rate and SpO_2 readings. These readings are stored with timestamps and linked to potential alerts when abnormal values are detected. Users can define personalized thresholds for heart rate and oxygen levels, and acknowledge alerts through the system.

This relational design ensures data integrity, traceability, and scalability across all components of the IoT-based health monitoring system.

9.API Functions

The HeartLink backend provides a set of secure RESTful APIs to manage the flow of data and control commands between the embedded hardware (ESP32) and the user interfaces (Web UI).

9.1 Data Ingestion API (Device-to-Cloud)

This API is used exclusively by the ESP32 device to securely transmit collected physiological data to the cloud.

Parameter	Specification
Endpoint	/ingest
Method	POST
Authentication	Required. JWT validated by API Gateway before reaching the backend.
Function	Receives raw PPG data, validates the payload, processes the data using HR/AC configuration, stores it in the Operational DB, and publishes it to Pub/Sub for analytics.
Request Body (JSON)	user_id: String (Unique identifier for the device owner) timestamp: String (ISO 8601 format) ppg_ir_raw: Array of Integers (Raw infrared readings) ppg_red_raw: Array of Integers (Raw red light readings)
Response (201 Created)	{"status": "success", "message": "Data ingested and published.", "record_id": "UUID"}
Error Codes	401 Unauthorized (Invalid/Expired JWT) 400 Bad Request (Invalid JSON payload/Missing fields)

Table 18: Data Ingestion API

9.2 Device Control API (Device Polling)

This API allows the ESP32 device to poll the cloud for the latest operational status (e.g., start or stop data collection) set by the user via the Web UI.

Parameter	Specification

Endpoint	/control
Method	GET
Authentication	Required. JWT validated to ensure the device is authorized.
Function	Retrieves the current desired operational state (set by /set_mode) for the specific user_id associated with the device's JWT.
Request Body (JSON)	None (User identity derived from JWT).
Response (201 Created)	{"command": "start"} or {"command": "stop"}
Error Codes	401 Unauthorized (Invalid/Expired JWT) 404 Not Found (No control state set for this user/device)

Table 19: Device Control API

9.3 User Control API (Frontend-to-Cloud)

This API is called by the Web UI to allow the user to change the desired operational state of their associated ESP32 device.

Parameter	Specification
Endpoint	/set_mode
Method	POST or PUT
Authentication	Required. JWT from Firebase JS SDK (validated against user's session).
Function	Updates the desired control command (start or stop) in a temporary storage/Operational DB that the /control API retrieves.
Request Body (JSON)	command: String (Value must be "start" or "stop")
Response (201 Created)	{"status": "success", "message": "Device command updated to [start/stop]."}
Error Codes	401 Unauthorized

	400 Bad Request (Invalid command value)

Table 20: User Control API

10. Test Cases

This section outlines the testing strategy employed to ensure the HeartLink system meets its functional, performance, and security requirements across all deployment layers. The testing approach adheres to established quality assurance practices, utilizing Unit Testing (UT), Integration Testing (IT), and System Testing (ST) to verify the integrity of the architecture from the embedded hardware to the user-facing analytics dashboards.

10.1 Scope and Methodology

The testing scope covers three primary layers of the HeartLink architecture:

- Hardware Layer:
  Data acquisition accuracy, embedded logic integrity, and secure data transmission protocols.
- Cloud Layer (Backend):
  API function correctness, security controls (Authentication/Authorization), data persistence, processing logic, and pipeline reliability.
- Presentation Layer:
  User experience, data rendering accuracy, system interactivity, and end-to-end data flow validation.

10.2 Test Case Classification

Testing was systematically organized into three tiers to ensure comprehensive coverage:

- Unit Testing (UT):
  Focuses on the smallest testable parts of the application, such as individual functions, methods, and UI components. In the Cloud Layer, UT verifies business logic (e.g., heart rate calculation) and data validation routines. In the Presentation Layer, UT confirms correct component rendering based on props and state.
- Integration Testing (IT):
  Verifies the communication and data exchange between different system components. Key IT focuses include:
  - Hardware-to-Cloud: Testing the secure HTTPS/JWT communication between the ESP32 and the API Gateway.
  - Backend-to-DB: Ensuring data is correctly written to the Operational DB and published to Pub/Sub.
  - Frontend-to-Backend: Validating API calls initiated by the Web UI (e.g., user login, control command issuance).
- System Testing (ST):

verifies the complete, fully integrated system against the specified requirements. ST covers crucial aspects like:

- Security: Testing horizontal authorization (User A cannot see User B's data).
- Performance: Load testing the /ingest API to verify Cloud Run's auto-scaling capability.
- Data Integrity: Verifying that data visualized on the Looker Studio dashboard accurately reflects the data stored in BigQuery.

10.3 Test Cases

5.1.1 Unit Tests

Test Case ID	Test Objective	Test Case Description (Pre-condition, Step, Result)	Actual Result	Status (Pass/Fail)
TC_UT_001	Sensor Data Accuracy	**Pre-condition:** MAX30102 driver is initialized. A known, stable PPG signal is fed to the sensor. **Test Step:** Read the raw ADC counts for IR and Red channels. **Expected Result:** The raw count values are within the documented 5% tolerance range for the known input signal.	80000~120000 (depends on the distance of the finger and sensor)	Pass
TC_UT_002	Business Logic Calculation	**Pre-condition:** A Python function calculate_heart_rate(ppg_data) is defined. Pre-calculated raw ppg_data that should yield a heart rate of 75 BPM is prepared. **Test Step:** Call the function: result = calculate_heart_rate(pre_calculated_data). **Expected Result:** The result is exactly 75.	75	Pass
TC_UT_003	Input Data Validation	**Pre-condition:** The data validation function validate_ingestion_payload(payload) is defined. A payload is created where the user_id field is missing. **Test Step:** Call the function: validate_ingestion_payload(missing_user_id_payload). **Expected Result:** The function raises a specific ValidationError exception.	ValidationError	Pass
TC_UT_004	DB Query Builder	**Pre-condition:** A function build_user_results_query(user_id) is defined to construct a SQL statement.	Query is valid.	Pass

		Test Step: Call the function: query = build_user_results_query("user123"). **Expected Result:** The generated query string correctly contains the statement WHERE user_id = 'user123' and is syntactically valid SQL.		
TC_UT_005	"Start/Stop" Button State	**Pre-condition:** The Button component receives a monitoring_status prop set to 'running'. **Test Step:** Render the Button component. **Expected Result:** The button displays its color/style reflects the running state.	Button changes to running status.	Pass

Table 21: Unit Test Cases

5.1.2 Integrated Tests

Test Case ID	Test Objective	Test Case Description (Pre-condition, Step, Result)	Actual Result	Status (Pass/Fail)
TC_IT_001	Device Control Reception	**Pre-condition:** ESP32 is connected to Wi-Fi. The cloud control status is set to 'start' via the /set_mode API. **Test Step:** The ESP32 calls the /control API. **Expected Result:** The ESP32 successfully receives a JSON response containing {"command": "start"}, and the device's data collection module is activated.	Receive the signal from the cloud.	Pass
TC_ IT_002	Secure Data Ingestion	**Pre-condition:** ESP32 has a valid, unexpired JWT from Firebase Auth. The device has collected a sample data payload.	Validate successfully.	Pass

		Test Step: The ESP32 sends a POST request with the PPG JSON payload and the JWT in the Authorization header to the API Gateway's /ingest endpoint. **Expected Result:** The API Gateway validates the JWT, and the Cloud Run service returns an HTTP 200 OK or 201 Created status, and the data is acknowledged.		
TC_ IT_003	Ingestion Auth Failure	**Pre-condition:** A request is formulated with a syntactically correct but expired JWT in the Authorization header. **Test Step:** Send the POST request with the expired JWT to the API Gateway's /ingest endpoint. **Expected Result:** The API Gateway rejects the request with an HTTP 401 Unauthorized status code and the request is not forwarded to Cloud Run.	The API Gateway rejects the request.	Pass
TC_IT_004	Backend-to-DB Write	**Pre-condition:** The Operational DB (Firestore/Cloud SQL) is online and configured. The Backend is ready to process a valid ingestion request. **Test Step:** Send a valid /ingest request. Query the Operational DB immediately after the successful response. **Expected Result:** A new record corresponding to the ingested data is successfully found in the Operational DB with the correct timestamp and all required fields populated.	Create a new record.	Pass
TC_IT_005	Backend-to-Pub/Sub Reliability	**Pre-condition:** A valid ingestion request is processed by the Backend. **Test Step:** Immediately after the Backend writes to the Operational DB, check the corresponding Pub/Sub topic for a new message.	A message containing the relevant data payload is successfully published to the	Pass

		Expected Result: A message containing the relevant data payload is successfully published to the configured Pub/Sub topic.	configured Pub/Sub topic.	
TC_IT_006	Login and Token Acquisition	**Pre-condition:** A user is logged out of the Web UI. **Test Step:** User enters correct Firebase credentials and clicks 'Log In'. **Expected Result:** Firebase JS SDK successfully authenticates the user, the UI loads the main dashboard, and a valid Firebase ID Token is stored locally for subsequent API calls.	Login successfully.	Pass
TC_IT_007	Control Feature Interaction	**Pre-condition:** User is logged in. The device is currently stopped. **Test Step:** User clicks the "Start Monitoring" button on the Web UI. **Expected Result:** The UI sends a successful HTTP request to the /set_mode?cmd=start API, the button state changes to "Stop Monitoring", and the UI displays a confirmation message.	The UI sends a successful HTTP request to the /set_mode?cmd=start API, the button state changes to "Stop Monitoring", and the UI displays a confirmation message.	Pass

Table 22: Integrated Test Cases

5.1.3 System Tests

Test Case ID	Test Objective	Test Case Description (Pre-condition, Step, Result)	Actual Result	Status (Pass/Fail)
TC_ST_001	Connectivity Reliability	**Pre-condition:** The device is actively running and uploading data every T seconds.	The data upload successly.	Pass

		Test Step: Observe the device's connectivity and data upload success rate over a continuous 24-hour period. **Expected Result:** The data upload success rate is >= 99.5%, and no unexpected device reboots or loss of connection occur.		
TC_ST_002	Horizontal Authorization	**Pre-condition:** User A and User B have valid, distinct JWTs. User B's data is stored in the DB. **Test Step:** User A's client attempts to call the /results API with User A's JWT but requesting User B's specific data records. **Expected Result:** The Backend correctly identifies the JWT mismatch/unauthorized access and returns an HTTP 403 Forbidden or HTTP 401 Unauthorized error, preventing access to User B's data.	The Backend correctly identifies the JWT mismatch/unauthorized access and returns an HTTP 403 Forbidden or HTTP 401 Unauthorized error, preventing access to User B's data.	Pass
TC_ST_003	Data Consistency (ETL)	**Pre-condition:** 100 sample records have been successfully ingested, stored in the Operational DB, and published to Pub/Sub. **Test Step:** Wait for the ETL/Stream process to complete. Run a simple count query on the BigQuery Analytics DB table. **Expected Result:** The total number of records in the BigQuery table equals 100, and the aggregated metrics (e.g., average heart rate) match the Operational DB calculations.	The total number of records in the BigQuery table equals 100, and the aggregated metrics (e.g., average heart rate) match the Operational DB calculations.	Pass
TC_ST_004	Looker Studio	**Pre-condition:** Historical data for the last month is stored in BigQuery.	The chart loads successfully, and the displayed average value is	Pass

	Report Accuracy	**Test Step:** Open the Looker Studio Dashboard and view the "Monthly Average Heart Rate" chart. **Expected Result:** The chart loads successfully, and the displayed average value is mathematically consistent with a direct SQL aggregation query run on the BigQuery table.	mathematically consistent with a direct SQL aggregation query run on the BigQuery table.	
TC_ST_005	End-to-End Data Visualization	**Pre-condition:** A device is actively streaming data for 5 minutes to the Cloud Backend. **Test Step:** User navigates to the Web UI dashboard to view the real-time data. **Expected Result:** The UI successfully calls the /results API, and the displayed heart rate and blood oxygen data points are visible and update every 1-2 seconds.	The UI successfully calls the /results API, and the displayed heart rate and blood oxygen data points are visible and update every 1-2 seconds.	Pass

Table 23: System Test Cases

11. Coding Standards

Adopting a strict set of coding standards is essential for the HeartLink project to ensure long-term maintainability, reduce the risk of errors, and facilitate collaboration among development teams (firmware, backend, and frontend). The standards outlined below cover naming conventions, formatting, documentation, and best practices across the project's primary technology stacks.

11.1 General Principles

- Readability: Code must be clear, simple, and self-documenting. Prioritize clarity over cleverness.
- Consistency: All code must adhere to the same styling and naming conventions throughout the entire project.
- Modularity: Functions and classes should follow the Single Responsibility Principle (SRP). Functions should generally be short (ideally < 50 lines).

11.2 Naming Conventions

Consistent naming makes code predictable and easier to navigate.

11.3 Code Formatting

Use automated formatters to enforce style and reduce debate.

- Indentation: Use 4 spaces for indentation. Tabs are forbidden to ensure cross-editor consistency.
- Line Length: Maximum line length is 120 characters. Lines exceeding this must be wrapped thoughtfully.
- Imports: Imports must be grouped and sorted: Standard library first, third-party libraries next, and local/project modules last.

11.4 Documentation and Comments

- Inline Comments: Use sparingly to explain why a complex piece of code exists, not what it is doing. Comments should be concise and placed above the relevant block of code.
- API Endpoints: All Cloud Run API endpoints must be documented using OpenAPI/Swagger specifications (easily generated via FastAPI).

11.5 Language-Specific Standards

11.5.1 Python (Cloud Run Backend)

Standard	Description
Style Guide	Adhere strictly to PEP 8.
Type Hinting	All function signatures must use type hints (e.g., def ingest_data(data: dict) -> bool:).
Error Handling	Use the built-in logging module instead of print() for all runtime messages. Use try...except blocks for expected failure points, catching specific exceptions rather than broad Exception.

Table 24: Backend Language-Specific Standards

11.5.2 JavaScript/TypeScript (Web UI Frontend)

Standard	Description
Style Guide	Adhere to a standard ESLint configuration (e.g., Airbnb or Standard).
Type Hinting	State should be managed clearly (e.g., via React hooks or context). Avoid direct DOM manipulation where possible.
Error Handling	Always use async/await for handling Promises (API calls, authentication) instead of chained .then()/.catch() for better readability.

Table 25: Frontend Language-Specific Standards

11.5.3 C/C++ (ESP32 Firmware)

Standard	Description
Style Guide	Practice RAII (Resource Acquisition Is Initialization). Ensure dynamic memory allocation (malloc, new) is paired with corresponding deallocation (free, delete) to prevent memory leaks in the limited firmware environment.
Type Hinting	Functions should return specific error codes (enum types) instead of general boolean values for failure, facilitating specific debugging.
Error Handling	Use #pragma once or traditional include guards (#ifndef...#define...#endif) in all header files.

Table 26: Hardware Language-Specific Standards

12. Deployment Guide and Source Code

12.1 Deployment Diagram

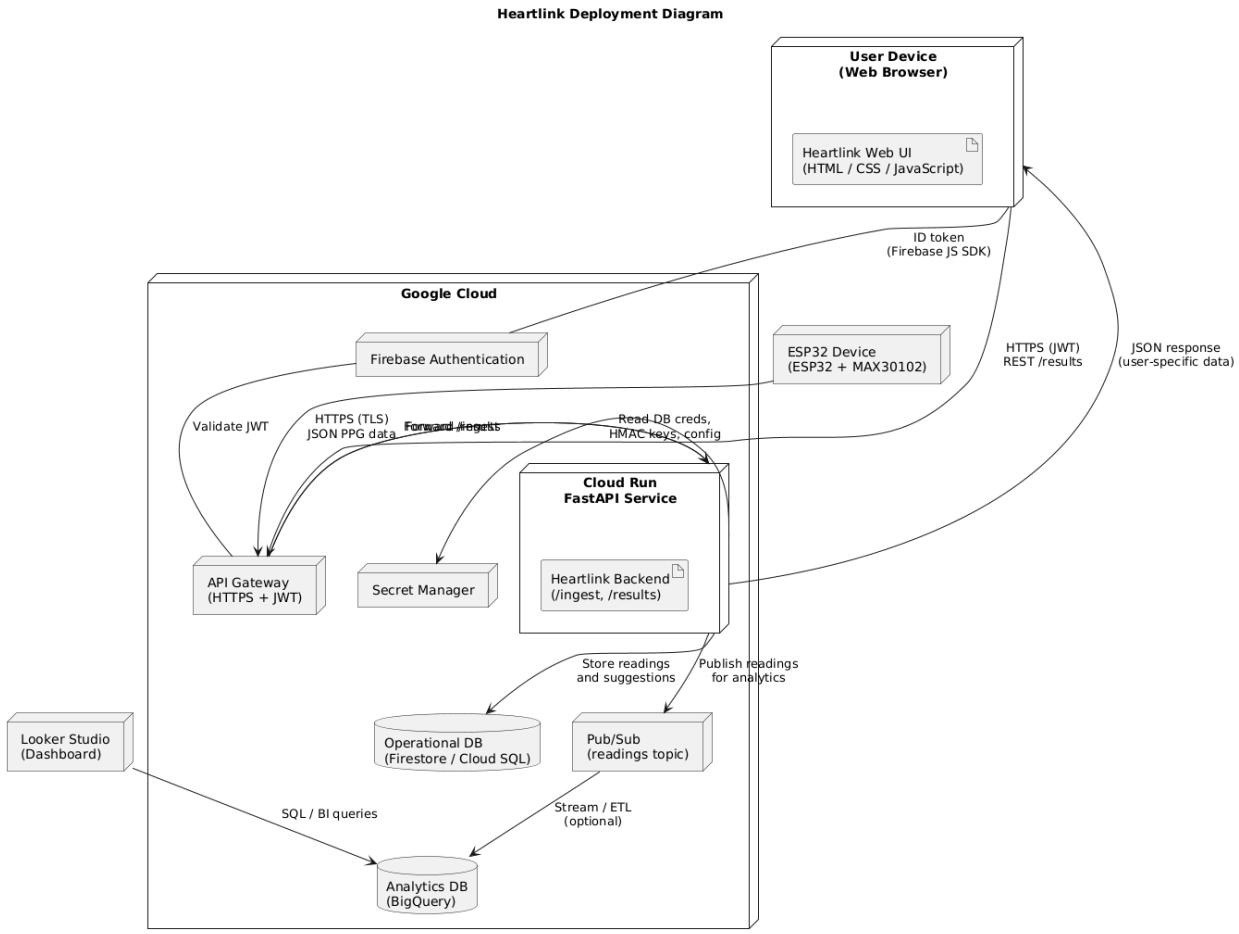

Figure 27: System Architecture

The HeartLink deployment outlines a robust, multi-layered architecture for secure physiological data collection and analysis.

The flow begins at the ESP32 Device, which collects physiological sensor data and securely transmits it to Google Cloud via HTTPS (TLS).

The cloud infrastructure is built around secure, serverless components. User and device identity is managed by Firebase Authentication. Data enters through an API Gateway (validating JWTs) before reaching the Heartlink Backend (Cloud Run Fast API Service). This backend handles ingestion, results calculation, and secure access to sensitive configuration via Secret Manager. Processed data is stored in the Operational DB (Firestore/Cloud SQL) and reliably published to

Pub/Sub for asynchronous delivery to the analytics pipeline. Ultimately, all data for business intelligence is consolidated in BigQuery.

For data visualization and user interaction, the system provides two avenues: a Web Browser UI for direct user-specific data retrieval (using HTTPS and JWTs), and Looker Studio for providing comprehensive analytics and real-time dashboards based on the BigQuery data. This design ensures clear separation of concerns, scalability, and security throughout the data lifecycle.

12.2 Deployment Guide

This guide outlines the sequential steps required to deploy the HeartLink system onto the Google Cloud Platform (GCP), covering foundational services, security setup, and application deployment.

12.2.1 Foundational Setup & Prerequisites

Before deploying the application code, the necessary GCP services must be enabled and fundamental resources created.

- Enable GCP APIs:
  In the Google Cloud Console, navigate to the "APIs & Services" dashboard and enable the following essential APIs: Cloud Run API, API Gateway API, BigQuery API, Cloud Pub/Sub API, Cloud SQL Admin API, and Secret Manager API.
- Set up Firebase:
  Create a new Firebase project and enable Firebase Authentication (required for user/device JWT generation). Note the project ID.
- Create Databases:
  Deploy your Operational DB (e.g., Cloud SQL or Firestore) and ensure your Analytics DB (BigQuery) tables (e.g., RAW_READINGS_TABLE) are created with the necessary schema.
- Configure Secrets:
  Use Secret Manager to securely store sensitive configurations, such as your database credentials, API keys, and HR/AC configuration values.
- Create Pub/Sub Topic:
  Create a dedicated Pub/Sub Topic (e.g., heartlink-ingestion-topic) for streaming raw data to the analytical pipeline.

12.2.2 Application Deployment (Cloud Run Backend)

This phase deploys the core business logic and API service.

- Containerize Backend:
  Package the Heartlink Backend code (Python/FastAPI) and all dependencies (including NumPy, PyTZ) into a Docker container image.

- Push Container Image:
  Push the container image to Google Artifact Registry (or Container Registry).
- Deploy Cloud Run Service:
  Deploy the container image to Cloud Run. Configure the service to be: * Publicly accessible (to be reached by API Gateway). * Minimum instances set to 1 (for fast response). * Service Account attached with permissions to access BigQuery, Pub/Sub, and Secret Manager.
- Verify Backend Access:
  Test the Cloud Run service's native URL to ensure it is running and responding (e.g., check a simple /health endpoint).

12.2.3 Security & Gateway Setup

This phase secures the public endpoint and enforces JWT validation.

- Create OpenAPI Spec:
  Write an OpenAPI (Swagger) configuration file (api-spec.yaml). This file must define: * The /ingest, /results, /control, and /set_mode paths. * The security definition pointing to your Firebase Authentication service for JWT validation. * The backend rule mapping the paths to your Cloud Run service URL.
- Deploy API Config:
  Deploy the OpenAPI specification to API Gateway to create the API configuration.
- Create API Gateway:
  Deploy a new API Gateway instance using the configuration created in 3.2. This generates the final, public HTTPS URL for your system.
- Test Security:
  Test the new API Gateway URL. Verify that requests without a valid JWT receive an HTTP 401 Unauthorized error, and requests with a valid JWT are successfully forwarded to Cloud Run.

12.2.4 Frontend and Hardware Integration

The final phase involves pointing client applications to the newly deployed services.

- Configure ESP32 Firmware:
  Update the ESP32 firmware code to use the API Gateway's public URL for all HTTPS communication (/ingest and /control). Ensure the firmware is using the correct Firebase SDK for secure JWT generation.
- Deploy Web UI:
  Deploy the Web UI (HTML/CSS/JavaScript) to a hosting service (e.g., Firebase Hosting or Cloud Storage). Update the JavaScript code to call the API Gateway URL for all backend interactions.

- Connect Looker Studio:
  Deploy the Web UI (HTML/CSS/JavaScript) to a hosting service (e.g., Firebase Hosting or Cloud Storage). Update the JavaScript code to call the API Gateway URL for all backend interactions.

12.3 Source Code of Hardware

https://github.com/Ivan-Kwan/Capstone/tree/master

13 Recommendations

12.2 Hardware Layer

- Implement Bi-directional Communication:
  - **Goal:** Replace the current HTTP polling mechanism in the ESP32's /control API with a persistent, low-power connection protocol like MQTT.
  - **Benefit:** Achieve true real-time device control (e.g., stopping data collection instantly) and significantly reduce the ESP32's power consumption and network overhead associated with continuous polling.
- Edge Data Pre-Processing and Filtering:
  - **Goal:** Enhance the ESP32 firmware to perform advanced signal filtering or preliminary calculations directly on the device before transmission.
  - **Benefit:** Reduce the volume of data sent over the network, lowering cloud costs, and ensuring that only high-quality, processed metrics reach the backend.
- Over-The-Air (OTA) Firmware Updates:
  - **Goal:** Implement a secure OTA update mechanism.
  - **Benefit:** Allow for remote deployment of bug fixes, security patches, and new features to devices already deployed in the field without requiring physical access.

12.3 Cloud Layer

- Real-time Anomaly Detection Stream:
  - **Goal:** Implement a serverless stream processing component (e.g., using Google Cloud Dataflow or a custom Pub/Sub subscriber) to monitor incoming PPG data against personalized, adaptive thresholds.
  - **Benefit:** Enable proactive alerts for critical health events (e.g., dangerous heart rate spikes) instead of relying solely on historical analysis.
- Formalize Machine Learning Pipeline (Vertex AI):
  - **Goal:** Leverage the historical data in BigQuery to train and deploy advanced Machine Learning models (using Vertex AI) for predictive health analytics (e.g., predicting stress levels or fitness improvements).
  - **Benefit:** Move the platform beyond basic monitoring to offer personalized, actionable health insights.

12.4 Front-end Layer

- Dedicated Mobile Application:
  - **Goal:** Develop native mobile applications (iOS/Android) or a high-quality Progressive Web App (PWA).

    - **Benefit:** Provide a superior, intuitive user experience, allow for push notifications for real-time alerts, and potentially integrate with platform-specific health APIs (e.g., Apple Health Kit).
- Enhanced Data Visualization Interactivity:
    - **Goal:** Upgrade the Web UI's data visualization libraries to support richer, more interactive charting features.
    - **Benefit:** Allow users to perform deeper self-service analysis of their physiological data directly in the browser.

14 Summary

Our goal is to advance smart physiological data management through the integration of cutting-edge IoT and cloud technologies. This project aims to increase data security, system scalability, and precision health monitoring in decentralized device applications. Building on the core principles of modern cloud architecture, our endeavor is dedicated to providing innovative solutions that support reliable data collection while also giving users more control and management over their health metrics. In order to address the difficulties that IoT operators experience in real-world deployment, it focuses on secure data pipelines, bridging the gap between embedded hardware communication and sophisticated cloud analytics.

Sensors for physiological data (MAX30102) and ESP32 microcontrollers for data processing are important parts of the system architecture. Software tools such as FastAPI (Python) for the cloud backend and JavaScript/HTML/CSS for the user interface are connected with the hardware components to facilitate data processing, secure API management, and the development of control algorithms. Secure wireless connections between components are facilitated by communication protocols like HTTPS/TLS and JWT (JSON Web Tokens).

In addition, the system has user interfaces for web applications that offer real-time data visualization, historical data analysis, and device status controls. APIs are also included for data ingestion and user-specific results retrieval. The process of the ESP32 securely transmitting data to the API Gateway, routing it through Cloud Run, storing it in the Operational DB, and publishing it to Pub/Sub for analysis in BigQuery is outlined in the deployment diagram.

15 References

MAX30102. (n.d.). https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf

Instructables. (2023, June 3). *Guide to Using MAX30102 Heart Rate and Oxygen Sensor With Arduino*. Instructables. https://www.instructables.com/Guide-to-Using-MAX30102-Heart-Rate-and-Oxygen-Sens/?utm_source=chatgpt.com

SmartDraw. (2019). *SmartDraw - Create Flowcharts, Floor Plans, and Other Diagrams on Any Device*. Smartdraw.com. https://www.smartdraw.com/

Espressif Documentation. (2025). Espressif.com. https://documentation.espressif.com/esp32_datasheet_cn.pdf

Figma. (2021). *Figma*. Figma. https://www.figma.com/community

(2025). Fanshaweonline.ca. https://www.fanshaweonline.ca/d2l/le/content/2001609/viewContent/17909532/View

All APIs and references. (2025). Google Cloud. https://cloud.google.com/pubsub/docs/apis

Node.js client library. (n.d.). Google Cloud. https://cloud.google.com/nodejs/docs/reference/iot/latest

APIs and reference. (2025). Google Cloud. https://cloud.google.com/bigquery/docs/reference

Welcome to Looker Studio! (2025). Google Cloud. https://cloud.google.com/looker/docs/studio

GeeksforGeeks. (2022, July 20). *Software Testing Boundary Value Analysis vs Equivalence Partitioning*. GeeksforGeeks. https://www.geeksforgeeks.org/software-testing/software-testing-boundary-value-analysis-vs-equivalence-partitioning/

BVA & EP Testing: Key Methods of Black box testing - Ficode. (2023, September 14). Ficode Technologies Limited. https://www.ficode.com/blog/boundary-value-analysis-bva-and-equivalence-partitioning-ep-testing

Pittet, S. (n.d.). *The different types of testing in Software*. Atlassian. https://www.atlassian.com/continuous-delivery/software-testing/types-of-software-testing

Doshi, K. (2023, March 22). *Types of Software Testing : Learn with Examples*. BrowserStack. https://www.browserstack.com/guide/types-of-testing

Team, A. (2024, September 19). *What is Functional Testing? Types and Example (Full Guide)*. AI-Powered End-To-End Testing | Applitools. https://applitools.com/blog/functional-testing-guide/?utm_source=chatgpt.com

Mozilla. (n.d.). *Web security*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/Security

Nano ESP32. (2024). Arduino.cc. https://docs.arduino.cc/hardware/nano-esp32